Nominal α-Unification

Guilherme Borges Brandão[†] Thomas Ammer[‡]
Daniele Nantes Sobrinho[†] Mauricio Ayala-Rinción[†]
Christian Urban[‡] Maribel Fernández[‡]
Mohammad Abdulaziz[‡]
[†] Universidade de Brasília, Brasília D.F., Brazil
[‡]King's College London, London, U.K.

March 9, 2026

Abstract

This theory verifies a non-deterministic procedure for nominal syntactic unification, i.e., nominal α-unification. The procedure is presented as a set of inductive rules, following Urban's seminal Isabelle formalisation in Isabelle [6, 5].

The formalisation is updated according to the PVS Rocha Oliveira et al. approach [2, 4], by proving the properties of symmetry, transitivity, and equivariance of the α-equivalence relation separately. This treatment of α-equivalence properties was also followed in the Coq formalisation by de Carvalho's et al. [1, 3]. In the PVS formalisation, nominal α-equivalence is presented as a functional recursive algorithm, proved sound and complete, from which executable code in Lisp can be extracted. In the Coq formalisation, a non-deterministic rule-based procedure was presented, following Urban's seminal approach, which is proved sound and complete; further, a recursive definition was presented, which is proved equivalent to the inductive procedure, and from which executable code was extracted.

Contents

theory *Swap*
imports *Main*
begin

fun *swapa* :: (*string* × *string*) ⇒ *string* ⇒ *string*
where

swapa (*b*,*c*) *a* = (*if b* = *a then c else* (*if c* = *a then b else a*))

fun *swapas*:: (*string* × *string*) *list* ⇒ *string* ⇒ *string*
where
swapas [] *a* = *a*
| *swapas* (*x*#*pi*) *a* = *swapa x* (*swapas pi a*)

lemma [*simp*]:
shows *swapas* [(*a*,*a*)] *b*=*b*
by *simp*

lemma *swapas-ab-ba*:
shows *swapas* [(*a*,*b*)] = *swapas* [(*b*,*a*)]
by *auto*

lemma *swapas-append*:
shows *swapas* (*pi1*@*pi2*) *a* = *swapas pi1* (*swapas pi2 a*)
by (*induct pi1*) *auto*

lemma *swapas-inv* [*simp*]:
shows *swapas* (*rev pi*) (*swapas pi a*) = *a*
apply(*induct-tac pi*)
apply(*auto simp add*: *swapas-append*)
done

lemma *swapas-rev-pi-a*:
assumes *swapas pi a* = *b*
shows *swapas* (*rev pi*) *b* = *a*
using *assms* **by** *auto*

lemma *swapas-rev-swapas-id* [*simp*]:
shows *swapas pi* (*swapas* (*rev pi*) *a*) = *a*
by (*metis swapas-rev-pi-a*)

lemma *swapas-rev-pi-b*:
assumes *swapas* (*rev pi*) *a* = *b*
shows *swapas pi b* = *a*
using *assms* **by** *auto*

lemma *swapas-comm*:
shows (*swapas* (*pi*@[(*a*,*b*)]) *c*) = (*swapas* ([(*swapas pi a*,*swapas pi b*)]@*pi*) *c*)
by (*metis swapa.simps swapas.simps*(*1*,*2*) *swapas-append swapas-rev-pi-a*)

end
theory *Terms*
imports *Swap*

begin

datatype *trm* = *Abst string trm*
| *Susp* (*string* × *string*) *list string*
| *Unit*
| *Atom string*
| *Paar trm trm*
| *Func string trm*

fun *swap* :: (*string* × *string*) *list* ⇒ *trm* ⇒ *trm*
where
swap pi (*Unit*) = *Unit*
| *swap pi* (*Atom a*) = *Atom* (*swapas pi a*)
| *swap pi* (*Susp pi′ X*) = *Susp* (*pi* @ *pi′*) *X*
| *swap pi* (*Abst a t*) = *Abst* (*swapas pi a*) (*swap pi t*)
| *swap pi* (*Paar t1 t2*) = *Paar* (*swap pi t1*) (*swap pi t2*)
| *swap pi* (*Func F t*) = *Func F* (*swap pi t*)

lemma *swap-id* [*simp*]:
shows *swap* [] *t* = *t*
by (*induct-tac t*) (*auto*)

lemma *swap-append*:
shows *swap* (*pi1*@*pi2*) *t* = *swap pi1* (*swap pi2 t*)
using *swapas-append* **by** (*induct t arbitrary*: *pi1 pi2*) *auto*

lemma *swap-empty*:
assumes *swap pi t* = *Susp* [] *X*
shows *pi* = []
using *assms swap.elims* **by** *blast*

fun *depth* :: *trm* ⇒ *nat*
where
depth (*Unit*) = *0*
| *depth* (*Atom a*) = *0*
| *depth* (*Susp pi X*) = *0*
| *depth* (*Abst a t*) = *1* + *depth t*
| *depth* (*Func F t*) = *1* + *depth t*
| *depth* (*Paar t t′*) = *1* + (*max* (*depth t*) (*depth t′*))

lemma
Suc-max-left: $Suc(max\ x\ y) = z \longrightarrow x < z$ **and**
Suc-max-right: $Suc(max\ x\ y) = z \longrightarrow y < z$
by(*auto*)

lemma *swap-depth* [*simp*]:
shows $depth\ (swap\ pi\ t) = depth\ t$
by (*induct t*) (*auto*)

fun *occurs* :: *string* ⇒ *trm* ⇒ *bool*
where
occurs X (*Unit*) = *False*
| *occurs X* (*Atom a*) = *False*
| *occurs X* (*Susp pi Y*) = (*if X* = *Y then True else False*)
| *occurs X* (*Abst a t*) = *occurs X t*
| *occurs X* (*Paar t1 t2*) = (*if* (*occurs X t1*) *then True else* (*occurs X t2*))
| *occurs X* (*Func F t*) = *occurs X t*

fun *vars-trm* :: *trm* ⇒ *string set*
where
vars-trm (*Unit*) = {}
| *vars-trm* (*Atom a*) = {}
| *vars-trm* (*Susp pi X*) = {*X*}
| *vars-trm* (*Paar t1 t2*) = (*vars-trm t1*)∪(*vars-trm t2*)
| *vars-trm* (*Abst a t*) = *vars-trm t*
| *vars-trm* (*Func F t*) = *vars-trm t*

lemma *vars-swap*:
shows *vars-trm* (*swap pi t*) = *vars-trm t*
by (*induct t*) *auto*

fun *sub-trms* :: *trm* ⇒ *trm set*
where
sub-trms (*Unit*) = {*Unit*}
| *sub-trms* (*Atom a*) = {*Atom a*}
| *sub-trms* (*Susp pi Y*) = {*Susp pi Y*}
| *sub-trms* (*Abst a t*) = {*Abst a t*} ∪ *sub-trms t*
| *sub-trms* (*Paar t1 t2*) = {*Paar t1 t2*} ∪ *sub-trms t1* ∪ *sub-trms t2*
| *sub-trms* (*Func F t*) = {*Func F t*} ∪ *sub-trms t*

fun *psub-trms* :: *trm* ⇒ *trm set*
where
psub-trms (*Unit*) = {}

| *psub-trms* (*Atom a*) = {}
| *psub-trms* (*Susp pi X*) = {}
| *psub-trms* (*Paar t1 t2*) = *sub-trms t1* ∪ *sub-trms t2*
| *psub-trms* (*Abst a t*) = *sub-trms t*
| *psub-trms* (*Func F t*) = *sub-trms t*

lemma *psub-sub-trms*:
assumes *t1* ∈ *psub-trms t2*
shows *t1* ∈ *sub-trms t2*
using *assms*
by(*induct t2*)(*auto*)

lemma *t-sub-trms-t*:
shows *t* ∈ *sub-trms t*
by (*induct t*) (*auto*)

lemma *abst-psub-trms*:
assumes *Abst a t1* ∈ *sub-trms t2*
shows *t1* ∈ *psub-trms t2*
using *assms*
apply(*induct t2 arbitrary*: *t1*)
apply(*auto simp add*: *t-sub-trms-t intro*: *psub-sub-trms*)
done

lemma *func-psub-trms*:
assumes *Func F t1* ∈ *sub-trms t2*
shows *t1* ∈ *psub-trms t2*
using *assms*
apply(*induct t2*)
apply(*auto simp add*: *t-sub-trms-t intro*: *psub-sub-trms*)
done

lemma *paar-psub-trms*:
assumes *Paar t1 t2* ∈ *sub-trms t3*
shows *t1* ∈ *psub-trms t3* **and** *t2* ∈ *psub-trms t3*
using *assms*
apply(*induct t3*)
apply(*auto simp add*: *t-sub-trms-t intro*: *psub-sub-trms*)
done

lemma *t-not-in-psub-trms-t*:
shows *t* ∉ *psub-trms t*
proof(*induct t*)
case (*Abst x1 t*)
then show *?case*
using *abst-psub-trms* **by** *auto*

next
case (*Susp x1 x2*)
then show *?case* **by** *simp*
next
case *Unit*
then show *?case* **by** *simp*
next
case (*Atom x*)
then show *?case* **by** *simp*
next
case (*Paar t1 t2*)
then have *Paar t1 t2* $\notin$ *sub-trms t1 Paar t1 t2* $\notin$ *sub-trms t2*
using *paar-psub-trms* **by** *auto*
then show *?case* **by** *simp*
next
case (*Func f t*)
then show *?case*
using *func-psub-trms* **by** *auto*
qed

lemma *if-sub-not-eq-then-psub*:
assumes *t1* $\in$ *sub-trms t2 t1* $\neq$ *t2*
shows *t1* $\in$ *psub-trms t2*
using *assms*
by (*induct t2*) *auto*

lemma *depth-psub-trms*:
assumes *t1* $\in$ *psub-trms t2*
shows *depth t1* $<$ *depth t2*
using *assms*
proof(*induct t2 arbitrary*: *t1*)
case (*Abst a t*)
then show *?case*
using *depth.simps*(*4*)[*of a t*]
psub-trms.simps(*5*)[*of a t*] *if-sub-not-eq-then-psub*
by *fastforce*
next
case (*Susp pi X*)
then show *?case* **by** *simp*
next
case *Unit*
then show *?case* **by** *simp*
next
case (*Atom a*)
then show *?case* **by** *simp*
next
case (*Paar t1' t2'*)
then show *?case*

```
    using depth.simps(6)[of t1' t2']
      psub-trms.simps(4) if-sub-not-eq-then-psub by fastforce
next
  case (Func f t1')
  then show ?case
    using depth.simps(5)[of f t1']
      psub-trms.simps(6)[of f t1'] if-sub-not-eq-then-psub
    by fastforce
qed

end

theory Atoms

imports Terms

begin

fun atms :: (string × string) list ⇒ string set where
atms [] = {} |
atms (x#xs) = ((atms xs) ∪  {fst(x),snd(x)})


lemma [simp]:
  atms (xs@ys) = atms xs ∪ atms ys
  by (induct xs) auto


lemma [simp]:
  atms (rev pi) = atms pi
  by (induct pi) auto


lemma a-not-in-atms:
  assumes a ∉ atms pi
  shows a = swapas pi a
  using assms by (induct pi) auto


lemma swapas-pi-ineq-a:
  assumes  swapas pi a ≠ a
  shows a ∈ atms pi
  using a-not-in-atms assms by force


lemma a-ineq-swapas-pi:
  assumes a ≠ swapas pi a
  shows a ∈ atms pi
  using a-not-in-atms assms by blast
```

lemma *swapas-pi-in-atms*:
assumes $a \in atms\ pi$
shows*swapas pi a* $\in$ *atms pi*
using *assms* **by** (*metis a-not-in-atms swapas-rev-pi-a*)

end

theory *Disagreement*

imports *Atoms*

begin

definition *ds* :: (*string* $\times$ *string*) *list* $\Rightarrow$ (*string* $\times$ *string*) *list* $\Rightarrow$ *string set* **where**
ds-def: *ds xs ys* $\equiv$ { *a* . *a* $\in$ (*atms xs* $\cup$ *atms ys*) $\wedge$ (*swapas xs a* $\neq$ *swapas ys a*) }

lemma
ds-elem: $[\![$*swapas pi a*$\neq$*a*$]\!] \Longrightarrow$*a*$\in$*ds* [] *pi*
using *ds-def swapas-pi-ineq-a* **by** *simp*

corollary *ds-elem-cp*: *a* $\notin$ *ds* [] *pi* $\Longrightarrow$ *swapas pi a* = *a*
using *ds-elem* **by** *blast*

lemma
elem-ds: $[\![$*a*$\in$*ds* [] *pi*$]\!] \Longrightarrow$*a*$\neq$*swapas pi a*
using *ds-def* **by** *simp*

lemma
ds-sym: *ds pi1 pi2* = *ds pi2 pi1*
using *ds-def* **by** *auto*

lemma
ds-trans: *c* $\in$ *ds pi1 pi3* $\Longrightarrow$ (*c* $\in$ *ds pi1 pi2* $\vee$ *c* $\in$ *ds pi2 pi3*)
using *ds-def a-not-in-atms swapas-pi-ineq-a* **by** *auto*

lemma *ds-cancel-pi-left*:

assumes (*c∈ ds* (*pi1@pi*) (*pi2@pi*))
shows (*swapas pi c* ∈ *ds pi1 pi2*)
using *assms ds-def swapas-append a-ineq-swapas-pi a-not-in-atms*
by (*metis* (*mono-tags*, *lifting*) *Un-iff mem-Collect-eq*)

lemma *ds-cancel-pi-right*:
(*swapas pi c∈ ds pi1 pi2*) ⟹ (*c∈ ds* (*pi1@pi*) (*pi2@pi*))
apply(*simp only*: *ds-def*)
apply(*auto*)
apply(*simp-all add*: *swapas-append*)
apply(*rule a-ineq-swapas-pi*,*clarify*,
drule a-not-in-atms,*drule a-not-in-atms*,*simp*)+
done

lemma *ds-equality*:
(*ds* [] *pi*)−{*a*,*swapas pi a*} = (*ds* [] ((*a*,*swapas pi a*)#*pi*))−{*swapas pi a*}
using *ds-def* **by** *auto*

lemma *ds-7*:
⟦*b*≠ *swapas pi b*;*a*∈*ds* [] ((*b*,*swapas pi b*)#*pi*)⟧⟹*a*∈*ds* [] *pi*
using *ds-def swapas-pi-in-atms a-ineq-swapas-pi swapas-rev-pi-a*
ds-elem elem-ds swapa.simps swapas.simps(*2*)
by *metis*

lemma *ds-cancel-pi-front*:
ds (*pi@pi1*) (*pi@pi2*) = *ds pi1 pi2*
apply(*simp only*: *ds-def*)
apply(*auto*)
apply(*simp-all add*: *swapas-append*)
apply(*rule swapas-pi-ineq-a*, *clarify*, *drule a-not-in-atms*, *simp*)+
apply(*drule swapas-rev-pi-a*, *simp*)+
done

lemma *ds-rev-pi-pi*:
ds ((*rev pi1*)*@pi1*) *pi2* = *ds* [] *pi2*
apply(*simp only*: *ds-def*)
apply(*auto*)
apply(*simp-all add*: *swapas-append*)
apply(*drule a-ineq-swapas-pi*, *assumption*)+
done

lemma *ds-rev*:

ds [] ((*rev pi1*)@*pi2*) = *ds pi1 pi2*
using *ds-cancel-pi-front ds-rev-pi-pi* **by** *blast*

lemma *ds-acabbc*:
⟦*a*≠*b*;*b*≠*c*;*c*≠*a*⟧⟹*ds* [(*a*, *b*), (*b*, *c*)] [(*a*, *c*)] = {*a*, *b*}
using *ds-def* **by** *auto*

lemma *ds-baab*:
⟦*a*≠*b*⟧⟹*ds* [(*b*,*a*)] [(*a*, *b*)] = {}
using *ds-def* **by** *auto*

lemma *ds-baab-id*:
⟦*a*≠*b*⟧⟹*ds* ([(*b*,*a*)]@[(*a*, *b*)]) [] = {}
using *ds-def ds-rev ds-baab* **by** *simp*

lemma *ds-abab*:
⟦*a*≠*b*⟧⟹*ds* [] [(*a*, *b*), (*a*, *b*)] = {}
using *ds-def* **by** *auto*

lemma *ds-comm*:
ds (*pi* @ [(*a*,*b*)]) ([(*swapas pi a*, *swapas pi b*)] @ *pi*) = {}
using *swapas-comm ds-def* **by** *simp*

lemma *ds-rev-pi-id*:
ds (*rev pi* @ *pi*) [] = {}
using *ds-rev-pi-pi*[*of pi* ‹[]›] *elem-ds*
ds-sym[*of* ‹(*rev pi* @ *pi*)› ‹[]›] **by** *fastforce*

lemma *ds-pi-rev-id*:
ds (*pi* @ *rev pi*) [] = {}
using *ds-rev-pi-id*[*of* ‹*rev pi*›] *rev-rev-ident*[*of pi*]
by *simp*

lemma *ds-swapas-eq*:
ds pi1 pi2 = {} ⟹ *swapas pi1 a* = *swapas pi2 a*
using *ds-elem*[*of* ‹*rev pi1* @ *pi2*›] *ds-rev*[*of pi1 pi2*]
empty-iff[*of a*] *swapas-append*[*of* ‹*rev pi1*› *pi2 a*] *swapas-inv* **by** *metis*

fun *flatten* :: (*string* × *string*)*list* ⇒ *string list* **where**
flatten [] = [] |
flatten (*x*#*xs*) = (*fst x*)#(*snd x*)#(*flatten xs*)

definition *ds-list* :: (*string* × *string*)*list* ⇒ (*string* × *string*)*list* ⇒ *string list* **where**
ds-list-def: *ds-list pi1 pi2* ≡ *remdups* ([*x*. *x* <− (*flatten* (*pi1*@*pi2*)), *x*∈*ds pi1*

pi2])

lemma *set-flatten-eq-atms*:
set (*flatten pi*) = *atms pi*
by (*induct pi*) *auto*

lemma *ds-list-equ-ds*:
set (*ds-list pi1 pi2*) = *ds pi1 pi2*
using *ds-list-def ds-def set-flatten-eq-atms* **by** *auto*

end
theory *Fresh*

imports *Disagreement*

begin

type-synonym *fresh-envs* = (*string* × *string*) *set*

inductive *fresh* :: *fresh-envs* ⇒ *string* ⇒ *trm* ⇒ *bool* (- ⊢ - ♯ - [*80*,*80*,*80*] *80*)
where
fresh-abst-ab[*intro*!]: ⟦*nabla* ⊢ *a* ♯ *t*; *a*≠*b*⟧ ⟹ *nabla* ⊢ *a* ♯ *Abst b t* |
fresh-abst-aa[*intro*!]: *nabla* ⊢ *a* ♯ *Abst a t* |
fresh-unit[*intro*!]: *nabla* ⊢ *a* ♯ *Unit* |
fresh-atom[*intro*!]: *a* ≠ *b* ⟹ *nabla* ⊢ *a* ♯ *Atom b* |
fresh-susp[*intro*!]: (*swapas* (*rev pi*) *a*,*X*) ∈ *nabla* ⟹ *nabla* ⊢ *a* ♯ *Susp pi X* |
fresh-paar[*intro*!]: ⟦*nabla* ⊢ *a* ♯ *t1*; *nabla* ⊢ *a* ♯ *t2*⟧ ⟹ *nabla* ⊢ *a* ♯ *Paar t1 t2* |
fresh-func[*intro*!]: *nabla* ⊢ *a* ♯ *t* ⟹ *nabla* ⊢ *a* ♯ *Func F t*

inductive-cases *Fresh-elims*:
nabla ⊢ *a* ♯ *Abst b t*
nabla ⊢ *a* ♯ *Unit*
nabla ⊢ *a* ♯ *Atom b*
nabla ⊢ *a* ♯ *Susp pi X*
nabla ⊢ *a* ♯ *Paar t1 t2*
nabla ⊢ *a* ♯ *Func F t*

lemma *fresh-swap-eqvt*:
assumes *nabla* ⊢ *a* ♯ *t*
shows *nabla* ⊢ *swapas pi a* ♯ *swap pi t*
using *assms*
by (*induct arbitrary*: *pi rule*: *fresh.induct*)
(*auto simp add*: *swapas-append dest*: *swapas-rev-pi-a*)

lemma *fresh-swap-left*:

```
  assumes nabla ⊢ a ♯ swap pi t
  shows nabla ⊢ swapas (rev pi) a ♯ t
  using assms
proof(induct t arbitrary: a pi)
  case (Abst x1 t)
  have nabla ⊢ a ♯ swap pi (Abst x1 t) by fact
  then have (nabla ⊢ a ♯ swap pi t ∧ a ≠ swapas pi x1) ∨ (a = swapas pi x1)
    by (metis Fresh-elims(1) swap.simps(4))
  moreover {
    assume as: nabla ⊢ a ♯ swap pi t ∧ a ≠ swapas pi x1
    have nabla ⊢ swapas (rev pi) a ♯ Abst x1 t
      using Abst.hyps as by auto
  }
  moreover {
    assume as: a = swapas pi x1
    then have nabla ⊢ swapas (rev pi) a ♯ Abst x1 t
      by (simp add: fresh-abst-aa)
  }
  ultimately show nabla ⊢ swapas (rev pi) a ♯ Abst x1 t
    by argo
next
  case (Susp x1 x2)
  have nabla ⊢ a ♯ swap pi (Susp x1 x2) by fact
  have (swapas (rev x1) (swapas (rev pi) a), x2) ∈ nabla
    by (metis Fresh-elims(4) Susp rev-append swap.simps(3) swapas-append)
  then show nabla ⊢ swapas (rev pi) a ♯ Susp x1 x2
    by (simp add: fresh-susp)
next
  case (Atom x)
  have nabla ⊢ a ♯ swap pi (Atom x) by fact
  then have swapas pi x ≠ a by (auto elim: Fresh-elims)
  then have x ≠ swapas (rev pi) a using swapas-rev-pi-b by blast
  then show nabla ⊢ swapas (rev pi) a ♯ Atom x
    by (simp add: fresh-atom)
qed (auto elim: Fresh-elims)



lemma fresh-swap-right:
  assumes nabla ⊢ swapas (rev pi) a ♯ t
  shows nabla ⊢ a ♯ swap pi t
  using assms fresh-swap-eqvt[of nabla ‹swapas (rev pi) a› t pi]
    swapas-rev-swapas-id[of pi a] by simp

lemma fresh-weak:
  assumes nabla1 ⊢ a ♯ t
  shows (nabla1 ∪ nabla2) ⊢ a ♯ t
  using assms
  by(induct rule: fresh.induct)(auto)
```

lemma *ds-empty-fresh-1*:
assumes *ds pi1 pi2 = {}*
shows *nabla ⊢ swapas pi1 a ♯ t ⟹ nabla ⊢ swapas pi2 a ♯ t*
using *assms ds-swapas-eq* **by** *simp*

lemma *ds-empty-fresh-2*:
assumes *ds pi1 pi2 = {}*
shows *nabla ⊢ a ♯ swap pi1 t ⟹ nabla ⊢ a ♯ swap pi2 t*
using *assms*
proof−
assume *assm1*: *nabla ⊢ a ♯ swap pi1 t*
have *i*: *swapas (rev pi1) a = swapas (rev pi2) a*
using *assms ds-swapas-eq swapas-rev-pi-a* **by** *metis*
with *assms* **have** *ii*: *nabla ⊢ swapas (rev pi2) a ♯ t*
using *fresh-swap-left*[*OF assm1*] *i* **by** *simp*
show *?thesis*
using *fresh-swap-right*[*OF ii*] **by** *simp*
qed

end
theory *PreEqu*

imports *Fresh*

begin

inductive *equ* :: *fresh-envs ⇒ trm ⇒ trm ⇒ bool* (- ⊢ - ≈ - [*80*,*80*,*80*] *80*) **where**
equ-abst-ab[*intro*!]: ⟦*a≠b*;(*nabla ⊢ a ♯ t2*);(*nabla ⊢ t1 ≈ (swap* [(*a*,*b*)] *t2*))⟧
⟹ (*nabla ⊢ Abst a t1 ≈ Abst b t2*) |
equ-abst-aa[*intro*!]: (*nabla ⊢ t1 ≈ t2*) ⟹ (*nabla ⊢ Abst a t1 ≈ Abst a t2*) |
equ-unit[*intro*!]: (*nabla ⊢ Unit ≈ Unit*) |
equ-atom[*intro*!]: *a=b⟹nabla ⊢ Atom a ≈ Atom b* |
equ-susp[*intro*!]: (∀ *c* ∈ *ds pi1 pi2*. (*c*,*X*) ∈ *nabla*) ⟹ (*nabla ⊢ Susp pi1 X ≈ Susp pi2 X*) |
equ-paar[*intro*!]: ⟦(*nabla ⊢ t1≈t2*);(*nabla ⊢ s1≈s2*)⟧ ⟹ (*nabla ⊢ Paar t1 s1 ≈ Paar t2 s2*) |
equ-func[*intro*!]: (*nabla ⊢ t1 ≈ t2*) ⟹ (*nabla ⊢ Func F t1 ≈ Func F t2*)

inductive-cases *Equ-elims*:
nabla ⊢ Atom a ≈ Atom b
nabla ⊢ Unit ≈ Unit
nabla ⊢ Susp pi1 X ≈ Susp pi2 X
nabla ⊢ Paar s1 t1 ≈ Paar s2 t2
nabla ⊢ Func F t1 ≈ Func F t2
nabla ⊢ Abst a t1 ≈ Abst a t2
nabla ⊢ Abst a t1 ≈ Abst b t2

lemma *equ-depth*:
assumes *nabla* ⊢ *t1* ≈ *t2*
shows *depth t1* = *depth t2*
using *assms* **by** (*rule equ.induct*, *simp-all*)

lemma *rev-pi-pi-equ*: (*nabla* ⊢ *swap* (*rev pi*) (*swap pi t*) ≈ *t*)
proof(*induct t*)
case (*Susp pi′ X*)
then show *?case*
using *ds-cancel-pi-left*[*of* - ‹*rev pi* @ *pi*› - ‹[]›]
ds-rev-pi-pi elem-ds ds-rev-pi-id **by** *auto*
qed (*auto*)

lemma *equ-pi-right*:
assumes ∀ *a* ∈ *ds* [] *pi*. *nabla* ⊢ *a* ♯ *t*
shows *nabla* ⊢ *t* ≈ *swap pi t*
using *assms*
proof(*induct t arbitrary*: *pi*)
case (*Abst b t′*)
have *swapas pi b* = *b* ∨ *swapas pi b* ≠ *b* **by** *simp*
moreover
{ assume *eq*: *swapas pi b* = *b*
have *nabla* ⊢ *Abst b t′* ≈ *Abst b* (*swap pi t′*)
apply (*rule equ-abst-aa*)
apply (*rule Abst.hyps*)
apply (*rule ballI*)
subgoal for *a*
apply (*rule Fresh-elims*(*1*)[*of nabla a b t′*])
using *Abst.prems elem-ds*[*of a pi*] *eq* **by** *auto*
done
then have *nabla* ⊢ *Abst b t′* ≈ *swap pi* (*Abst b t′*) **using** *eq* **by** *simp*
}
moreover
{ assume *neq*: *b* ≠ *swapas pi b*
obtain *c* **where** *c-def*: *c* = *swapas pi b* **by** *simp*
have *c* ∈ *ds* [] *pi*
by (*metis append.right-neutral c-def ds-cancel-pi-front ds-cancel-pi-left ds-elem neq*)
then have *one*: *nabla* ⊢ *c* ♯ *Abst b t′* **using** *assms Abst.prems* **by** *auto*
have *two*: *b* ≠ *c* **using** *neq c-def* **by** *blast*
have *nabla* ⊢ *c* ♯ *t′* **using** *one two* **by** *cases auto*
have *nabla* ⊢ *Abst b t′* ≈ *Abst c* (*swap pi t′*)
proof(*rule equ-abst-ab*)
show *b* ≠ *c* **using** *neq c-def* **by** *blast*
have *b-is-swap*: *b* = *swapas* (*rev pi*) *c* **using** *c-def* **by** *simp*
show *nabla* ⊢ *b* ♯ *swap pi t′*

```
    using Abst.prems Fresh-elims(1) ds-elem fresh-swap-right neq swapas-rev-pi-a
by metis
    show nabla ⊢ t' ≈ swap [(b, c)] (swap pi t')
    proof −
      have fresh-ext:
       ∀ a ∈ ds [] ((b,c) # pi). nabla ⊢ a ♯ t'
      proof (rule ballI)
       fix a assume A: a ∈ ds [] ((b,c)#pi)
       then have a = c ∨ a ∈ ds [] pi
        using c-def ds-7 neq by auto
       then show nabla ⊢ a ♯ t'
       proof
        assume a = c
        with ‹nabla ⊢ c ♯ t'› show ?thesis by simp
       next
        assume a ∈ ds [] pi
        show ?thesis
        proof(rule Fresh-elims(1)[of nabla a b t'], goal-cases)
         case 1
         then show ?case
          using Abst.prems ‹a ∈ ds [] pi›
         by simp
        next
         case 2
         then show ?case by simp
        next
         case 3
         have a ≠ swapas ((b, c) # pi) a
          using A elem-ds
          by blast
         hence a ≠ b using c-def A
          by (auto simp add: if-split)
         hence False using 3 by simp
         then show ?case by simp
       qed
      qed
     qed
      from Abst.hyps[OF fresh-ext]
      have nabla ⊢ t' ≈ swap ([(b,c)]@pi) t'
       by simp
      moreover have swap ([(b,c)]@pi) t' = swap [(b,c)] (swap pi t')
       using swap-append by blast
      ultimately show ?thesis by simp
     qed
    qed
   then have nabla ⊢ Abst b t' ≈ swap pi (Abst b t') using neq c-def by force
}

 ultimately show ?case by argo
```

next
case (*Susp pi′ X*)
have ∀ *a*∈*ds* [] *pi*. *nabla* ⊢ *a* ♯ *Susp pi′ X* **by** *fact*
then have *nabla* ⊢ *Susp pi′ X* ≈ *Susp* (*pi* @ *pi′*) *X*
using *Fresh-elims*(*4*) *equ-susp ds-def ds-cancel-pi-left*
by (*metis* (*lifting*) *append-self-conv2 swapas-rev-pi-a*)
then show *?case* **by** *simp*
next
case *Unit*
then show *?case*
using *equ-unit swap.simps*(*2*) **by** *force*
next
case (*Atom b*)
then show *?case*
apply *simp*
apply (*rule equ-atom*)
using *Fresh-elims*(*3*) *ds-elem-cp*
by *metis*
next
case (*Paar t1 t2*)
then have *hypsp1*: (∀ *a*∈*ds* [] *pi*. *nabla* ⊢ *a* ♯ *t1*)
and *hypsp2*: (∀ *a*∈*ds* [] *pi*. *nabla* ⊢ *a* ♯ *t2*)
using *Paar.prems Fresh-elims*(*5*)
apply *meson*
using *Paar.prems Fresh-elims*(*5*) **by** *blast*
have *nabla* ⊢ *Paar t1 t2* ≈ *Paar* (*swap pi t1*) (*swap pi t2*)
apply(*rule equ-paar*)
apply(*rule Paar.hyps*)
apply (*simp add*: *hypsp1*)
by (*simp add*: *Paar.hyps*(*2*) *hypsp2*)
then show *?case* **by** *simp*
next
case (*Func f t′*)
then have *hypsf*: ∀ *a*∈*ds* [] *pi*. *nabla* ⊢ *a* ♯ *t′* **using** *fresh-func*
by (*metis Fresh-elims*(*6*))
have *nabla* ⊢ *Func f t′* ≈ *Func f* (*swap pi t′*)
apply(*rule equ-func*)
apply(*rule Func.hyps*)
using *hypsf* **by** *auto*
then show *?case*
by *simp*
qed

lemma *pi-comm*: *nabla* ⊢ (*swap* (*pi* @ [(*a*,*b*)]) *t*) ≈ (*swap* ([(*swapas pi a*, *swapas pi b*)] @ *pi*) *t*)
proof(*induct t*)
case (*Abst c t*)
then show *?case* **using** *swapas-comm* **by** *force*

next
case (*Susp* π' *X*)
have *rew1*:
swap (*pi* @ [(*a*,*b*)]) (*Susp* π' *X*) = *Susp* (*pi* @ [(*a*,*b*)] @ π') *X*
by *simp*
have *rew2*:
swap ([(*swapas pi a*, *swapas pi b*)] @ *pi*) (*Susp* π' *X*)
= *Susp* ([(*swapas pi a*, *swapas pi b*)] @ *pi* @ π') *X*
by *simp*
have *fresh*:
$\forall$ *c* $\in$ *ds* (*pi* @ [(*a*,*b*)] @ π') ([(*swapas pi a*, *swapas pi b*)] @ *pi* @ π'). (*c*, *X*) $\in$ *nabla*
using *swapas-append swapas-comm ds-def* **by** *simp*
from *rew1 rew2 fresh*
show *?case*
using *equ-susp* **by** *simp*
next
case *Unit*
then show *?case* **by** *force*
next
case (*Atom c*)
then show *?case*
using *equ-atom swapas-comm* **by** *auto*
next
case (*Paar t1 t2*)
then show *?case* **by** *force*
next
case (*Func f t*)
then show *?case* **by** *force*
qed

lemma *l3-jud*:
assumes (*nabla* $\vdash$ *t1* $\approx$ *t2*)
shows (*nabla* $\vdash$ *a* $\sharp$ *t1*) $\longrightarrow$ (*nabla* $\vdash$ *a* $\sharp$ *t2*)
using *assms*
proof(*induction rule*: *equ.induct*)
case (*equ-abst-ab b c nabla t2 t1*)
then show *?case*
using *fresh-swap-left Fresh-elims*(*1*) *fresh-abst-aa fresh-abst-ab rev-singleton-conv swapa.simps swapas.simps*(*1*,*2*)
by *metis*
next
case (*equ-abst-aa nabla t1 t2 b*)
then show *?case*
using *fresh-swap-left Fresh-elims*(*1*) *fresh-abst-aa fresh-abst-ab*
by *metis*
next
case (*equ-unit nabla*)

```
  then show ?case
    by blast
next
  case (equ-atom b c nabla)
  then show ?case
    by simp
next
  case (equ-susp pi1 pi2 X nabla)
  then show ?case
   using Fresh-elims(4) ds-elem-cp ds-rev fresh-susp swapas-append swapas-rev-pi-a
    by metis
next
  case (equ-paar nabla t1 t2 s1 s2)
  then show ?case
    using Fresh-elims(5) by blast
next
  case (equ-func nabla t1 t2 f)
  then show ?case
    using Fresh-elims(6) by blast
qed



lemma ds-empty-equiv-1:
  assumes ds pi1 pi2 = {}
  shows nabla ⊢ swap pi1 t1 ≈ t2 ⟹ nabla ⊢ swap pi2 t1 ≈ t2
  using assms
proof(induction t1 arbitrary: t2)
  case (Abst a t1')
  then obtain b t2'
    where t2: t2 = Abst b t2'
    by(auto elim: equ.cases)
  with Abst have i: nabla ⊢ Abst (swapas pi1 a) (swap pi1 t1') ≈ Abst b t2'
    using swap.simps(4) by simp
  then show ?case
  proof(cases ‹swapas pi1 a = b›)
    case True
    then have nabla ⊢ swap pi1 t1' ≈ t2'
      using Abst i Equ-elims(6) by blast
    then have ii: nabla ⊢ swap pi2 t1' ≈ t2'
      using Abst by simp
    then have nabla ⊢ Abst (swapas pi2 a) (swap pi2 t1') ≈ Abst b t2'
      using True ds-swapas-eq[OF Abst(3), of a] equ-abst-aa[OF ii, of ‹swapas pi2
a›]
      by simp
    then show ?thesis
      using t2 by simp
  next
```

```
    case False
    then have equ-ab-pi1: nabla ⊢ swap pi1 t1' ≈ swap [(swapas pi1 a, b)] t2'
      and fresh-ab-pi1: nabla ⊢ swapas pi1 a ♯ t2'
      using i Equ-elims(7)[of nabla ‹swapas pi1 a› ‹swap pi1 t1'› b t2']
      by blast+
    have equ-ab-pi2: nabla ⊢ swap pi2 t1' ≈ swap [(swapas pi2 a, b)] t2'
      using equ-ab-pi1 ds-swapas-eq[OF Abst(3), of a]
        Abst(3) Abst(1)[of ‹swap [(swapas pi1 a, b)] t2'›] by auto
    have fresh-ab-pi2: nabla ⊢ swapas pi2 a ♯ t2'
      using fresh-ab-pi1 ds-swapas-eq[OF Abst(3), of a] by simp
    with equ-ab-pi2 have nabla ⊢ Abst (swapas pi2 a) (swap pi2 t1') ≈ Abst b t2'
      using equ-abst-ab[OF False, of nabla t2'] ds-swapas-eq[OF Abst(3), of a] by
simp
    then show ?thesis using t2 by simp
  qed
next
  case (Susp pi X)
  then obtain pi'
    where t2: t2 = Susp pi' X
    by (auto elim:equ.cases)
  with Susp have i: nabla ⊢ Susp (pi1 @ pi) X ≈ Susp pi' X
    by simp
  then have ∀ c ∈ ds (pi1@pi) pi'. (c,X) ∈ nabla
    using Equ-elims(3)[OF i] by auto
  with Susp have ∀ c ∈ ds (pi2@pi) pi'. (c,X) ∈ nabla
    using ds-cancel-pi-left ds-sym ds-trans
    by blast
  then have nabla ⊢ Susp (pi2 @ pi) X ≈ Susp pi' X
    using equ-susp by simp
  then show ?case using t2 swap.simps(3) by simp
next
  case (Atom a)
  then show ?case
    using ds-swapas-eq[OF Atom(2), of a]
    by (auto elim: equ.cases)
qed (auto elim: equ.cases)


lemma ds-empty-equiv-2:
  assumes ds pi1 pi2 = {}
  shows nabla ⊢ t1 ≈ swap pi1 t2 ⟹ nabla ⊢ t1 ≈ swap pi2 t2
using assms
proof(induction t2 arbitrary: pi1 pi2 t1)
  case (Abst b t2')
  then obtain a t1'
    where t1: t1 = Abst a t1'
    by(auto elim: equ.cases)
  with Abst have i: nabla ⊢ Abst a t1' ≈ Abst (swapas pi1 b) (swap pi1 t2')
    using swap.simps(4) by simp
```

```
  then show ?case
  proof(cases ‹swapas pi1 b = a›)
    case True
    then have nabla ⊢ t1' ≈ swap pi1 t2'
      using Abst i Equ-elims(6) by blast
    then have ii: nabla ⊢ t1' ≈ swap pi2 t2'
      using Abst by simp
    then have nabla ⊢ Abst a t1' ≈ Abst (swapas pi2 b) (swap pi2 t2')
      using True ds-swapas-eq[OF Abst(3), of b] equ-abst-aa[OF ii, of ‹swapas pi2
b›]
      by simp
    then show ?thesis
      using t1 by simp
  next
    case False
    then have equ-ab: nabla ⊢ t1' ≈ swap [(a, swapas pi1 b)] (swap pi1 t2')
      and fresh-ab-pi1: nabla ⊢ a ♯ swap pi1 t2'
      using i Equ-elims(7) by blast+

    then have equ-ab-pi1: nabla ⊢ t1' ≈ swap ([(a, swapas pi1 b)]@ pi1) t2'
      using swap-append[of ‹[(a, swapas pi1 b)]› pi1 t2'] by simp

    have ds-empty: ds ([(a, swapas pi1 b)] @ pi1) ([(a, swapas pi2 b)] @ pi2) = {}
      using Abst(3) ds-swapas-eq[OF Abst(3), of b]
        ds-cancel-pi-front[of ‹[(a, swapas pi1 b)]› pi1 pi2] by simp

    hence nabla ⊢ t1' ≈ swap ([(a, swapas pi2 b)]@ pi2) t2'
      using Abst(1)[OF equ-ab-pi1 ds-empty] by simp
    hence equ-ab-pi2: nabla ⊢ t1' ≈ swap [(a, swapas pi2 b)] (swap pi2 t2')
      using swap-append[of ‹[(a, swapas pi2 b)]› pi2 t2'] by simp

    have fresh-ab-pi2: nabla ⊢ a ♯ swap pi2 t2'
      using ds-empty-fresh-2 Abst(3) fresh-ab-pi1 by auto
    with equ-ab-pi2 have nabla ⊢ Abst a t1' ≈ Abst (swapas pi2 b) (swap pi2 t2')
      using equ-abst-ab ds-swapas-eq False assms Abst(3) by auto
    then show ?thesis using t1 by simp
  qed
next
  case (Susp pi' X)
  then obtain pi
    where t1: t1 = Susp pi X
    by (auto elim:equ.cases)
  with Susp have i: nabla ⊢ Susp pi X ≈ Susp (pi1 @ pi') X
    by simp
  then have ∀ c ∈ ds pi (pi1 @ pi'). (c,X) ∈ nabla
    using Equ-elims(3)[OF i] by auto
  with Susp have ∀ c ∈ ds pi (pi2 @ pi'). (c,X) ∈ nabla
    using ds-cancel-pi-left ds-sym ds-trans
    by blast
```

```
  then have nabla ⊢ Susp pi X ≈ Susp (pi2 @ pi') X
    using equ-susp by simp
  then show ?case using t1 swap.simps(3) by simp
next
  case (Atom a)
  then show ?case
    using ds-swapas-eq by (auto elim: equ.cases)
qed (auto elim: equ.cases)


lemma equ-equivariance:
  assumes nabla ⊢ t1 ≈ t2
  shows nabla ⊢ swap pi t1 ≈ swap pi t2
  using assms
proof(induct rule: equ.induct)
  case (equ-abst-ab a b nabla t2' t1')
  have i: nabla ⊢ swapas pi a ♯ swap pi t2'
    using fresh-swap-eqvt[OF equ-abst-ab(2), of pi] by simp

  have nabla ⊢ swap pi t1' ≈ swap (pi @ [(a,b)]) t2'
    using equ-abst-ab(4) swap-append[of pi ‹[(a,b)]› t2'] by simp
  then have nabla ⊢ swap pi t1' ≈ swap ([(swapas pi a, swapas pi b)] @ pi) t2'
    using ds-empty-equiv-2[OF ds-comm[of pi a b]] by auto
  then have ii: nabla ⊢ swap pi t1' ≈ swap [(swapas pi a, swapas pi b)] (swap pi
t2')
    using ds-empty-equiv-2[OF ds-comm[of pi a b]]
      swap-append[of ‹[(swapas pi a, swapas pi b)]› pi t2'] by simp

  have iii: swapas pi a ≠ swapas pi b
    using equ-abst-ab.hyps(1) swapas-rev-pi-a by blast

  with i ii
  have nabla ⊢ Abst (swapas pi a) (swap pi t1') ≈ Abst (swapas pi b) (swap pi t2')
    using equ.equ-abst-ab by auto
  hence nabla ⊢ swap pi (Abst a t1') ≈ swap pi (Abst b t2')
    using swap.simps(4) by simp
  then show ?case by simp
next
  case (equ-abst-aa nabla t1' t2' a)
  then show ?case
    using equ.equ-abst-aa by simp
next
  case (equ-susp pi1 pi2 X nabla)
  then show ?case using ds-cancel-pi-front
    by auto
qed (auto)

lemma swap-inv-side:
  shows nabla ⊢ swap pi t1 ≈ t2 = nabla ⊢ t1 ≈ swap (rev pi) t2
```

proof

{assume *assm1*: *nabla* ⊢ *swap pi t1* ≈ *t2*
from *equ-equivariance*[*OF assm1*, *of* ‹*rev pi*›]
have *nabla* ⊢ *swap* (*rev pi* @ *pi*) *t1* ≈ *swap* (*rev pi*) *t2*
using *swap-append*[*of* ‹*rev pi*› *pi t1*] **by** *simp*
then show *nabla* ⊢ *t1* ≈ *swap* (*rev pi*) *t2*
using *ds-empty-equiv-1*[*OF ds-rev-pi-id*[*of pi*], *of nabla t1* ‹*swap* (*rev pi*) *t2*›]
swap-id[*of t1*] **by** *simp***}**

{assume *assm2*: *nabla* ⊢ *t1* ≈ *swap* (*rev pi*) *t2*
from *equ-equivariance*[*OF assm2*, *of pi*]
have *nabla* ⊢ *swap pi t1* ≈ *swap* (*pi* @ *rev pi*) *t2*
using *swap-append*[*of pi* ‹*rev pi*› *t2*] **by** *simp*
then show *nabla* ⊢ *swap pi t1* ≈ *t2*
using *ds-empty-equiv-2*[*OF ds-pi-rev-id*[*of pi*], *of nabla* ‹*swap pi t1*› *t2*]
swap-id[*of t2*] **by** *simp***}**
qed

lemma *equ-swap-abba*:
assumes *n* = *depth t1*
shows *nabla* ⊢ *swap* [(*a*,*b*)] *t1* ≈ *t2* ⟹ *nabla* ⊢ *t1* ≈ *swap* [(*b*,*a*)] *t2*
proof−
assume *nabla* ⊢ *swap* [(*a*,*b*)] *t1* ≈ *t2*
with *ds-baab*[*of b a*]
have *nabla* ⊢ *swap* [(*b*,*a*)] *t1* ≈ *t2*
using *ds-empty-equiv-1* **by** *blast*
then show *?thesis*
using *swap-inv-side*[*of nabla* ‹[(*b*,*a*)]›] **by** *simp*
qed

lemma *equ-equiv-pi*:
assumes ∀ *a* ∈ *ds pi1 pi2*. *nabla* ⊢ *a* ♯ *t*
shows *nabla* ⊢ *swap pi1 t* ≈ *swap pi2 t*
using *assms equ-pi-right*[*of* ‹*rev pi1* @ *pi2*› *nabla t*]
swap-inv-side[*of nabla pi1 t* ‹*swap pi2 t*›] *ds-rev swap-append* **by** *simp*

lemma *equ-symm*:
shows (*nabla* ⊢ *t1* ≈ *t2*) ⟹ (*nabla* ⊢ *t2* ≈ *t1*)
proof(*induction rule*: *equ.induct*)
case (*equ-abst-ab a b nabla t2′ t1′*)
have *i*: *nabla* ⊢ *b* ♯ *swap* [(*a*, *b*)] *t2′*
using *fresh-swap-eqvt*[*of nabla a t2′* [(*a*,*b*)]] *equ-abst-ab*(*2*) **by** *auto*
with *equ-abst-ab*(*4*)
have *b-fresh*: *nabla* ⊢ *b* ♯ *t1′*
using *l3-jud* **by** *blast*
from *equ-abst-ab*(*4*)
have *ii*: *nabla* ⊢ *t2′* ≈ *swap* [(*b*, *a*)] *t1′*

```
    using equ-swap-abba by simp
  show ?case
    using equ.equ-abst-ab[OF equ-abst-ab(1)[symmetric] b-fresh ii] by simp
next
  case (equ-abst-aa nabla t1' t2' a)
  then show ?case
    using equ.equ-abst-aa[OF equ-abst-aa(2), of a] by simp
next
  case (equ-unit nabla)
  then show ?case by auto
next
  case (equ-atom a b nabla)
  then show ?case by auto
next
  case (equ-susp pi1 pi2 X nabla)
  then show ?case
    using ds-sym by auto
next
  case (equ-paar nabla t1' t2' s1' s2')
  then show ?case by auto
next
  case (equ-func nabla t1' t2' f)
  then show ?case
    using equ.equ-func[OF equ-func(2), of f] by simp
qed


lemma equ-trans:
  assumes nabla ⊢ t1 ≈ t2 nabla ⊢ t2 ≈ t3
  shows nabla ⊢ t1 ≈ t3
  using assms
proof(induction ‹depth t1› arbitrary: t1 t2 t3 rule: nat-less-induct)
  case 1
  note IH = this
  have IH-usable:
    depth t1' < depth t1 ⟹ nabla ⊢ t1' ≈ t2' ⟹  nabla ⊢ t2' ≈ t3' ⟹ nabla
⊢ t1' ≈ t3'
    for t1' t2' t3' using IH by blast
  have nabla ⊢ t1 ≈ t2 ⟹ nabla ⊢ t2 ≈ t3 ⟹  nabla ⊢ t1 ≈ t3
  proof−
    assume t12: nabla ⊢ t1 ≈ t2 and t23: nabla ⊢ t2 ≈ t3
    show ?thesis
     proof(cases rule: equ.cases[OF ‹nabla ⊢ t1 ≈ t2›])
       case (1 a b nabla t2' t1')
       from 1(2) have depth t1': depth t1' < depth t1
         using depth.simps(4) by auto
       from t23 show ?thesis
       proof(cases rule: equ.cases)
         case (equ-abst-ab b c t3' t2')
```

```
  then show ?thesis
  proof(cases a = c)
    case True
    have i: nabla ⊢ swap [(c,b)] t2' ≈ swap ([(c,b)]@[(b,c)]) t3'
      using equ-equivariance[OF equu-abst-ab(5), of ‹[(c,b)]›] 1(1)
        swap-append[of ‹[(c,b)]› ‹[(b,c)]› t3'] by simp
    have ii: nabla ⊢ swap [(c,b)] t2' ≈ t3'
      using ds-empty-equiv-2[OF ds-baab-id[OF equ-abst-ab(3)] i] by simp
    with equ-abst-ab 1 True
    have iii: nabla ⊢ t1' ≈ swap [(c, b)] t2' by blast
    with ii have nabla ⊢ t1' ≈ t3'
      using IH-usable[OF deptht1, of ‹swap [(c, b)] t2'› t3'] 1(1)
      by simp
    then show ?thesis using True equ-abst-ab(2) 1(1,2) by auto
  next
    case False

    have deptht2: depth (swap [(a,b)] t2') < depth t1
      using depth.simps(4) equ-depth[OF 1(6)] swap-depth 1(3) deptht1
      equ-abst-ab(1) by simp

    from equ-abst-ab(1,4,5) 1(1,3,5)
      have ii: nabla ⊢ a ♯ swap [(b,c)] t3'
        using l3-jud by simp

    then have a-fresh-t3: nabla ⊢ a ♯ t3'
      using fresh-swap-left[OF ii] False equ-abst-ab 1 by simp
    have b-fresh-t3: nabla ⊢ b ♯ t3'
      using equ-abst-ab(4) 1(1) by simp

    from 1(1) equ-abst-ab(5)
    have nabla ⊢ swap [(a,b)] t2' ≈ swap ([(a,b)]@[(b,c)]) t3'
      using equ-equivariance[OF equ-abst-ab(5)]
     swap-append[of ‹[(a, b)]› ‹[(b,c)]› t3'] by simp
    then have iii: nabla ⊢ swap [(a,b)] t2' ≈ swap ([(a,b),(b,c)]) t3'
      by simp

    have ds [(a,b), (b,c)] [(a,c)] = {a,b}
      using ds-acabbc equ-abst-ab 1 False by simp
    with a-fresh-t3 b-fresh-t3
    have iv: nabla ⊢ swap [(a,b),(b,c)] t3' ≈ swap [(a,c)] t3'
      using equ-equiv-pi by simp

    with 1(1) iii have nabla ⊢ swap [(a,b)] t2' ≈ swap [(a,c)] t3'
      using IH-usable[OF deptht2, of ‹swap ([(a,b),(b,c)]) t3'› ‹swap [(a,c)]
t3'›]
      by simp

    with 1(1,3,6) have nabla ⊢ t1' ≈ swap [(a,c)] t3'
```

```
      using IH-usable[OF depth1, of ‹swap [(a, b)] t2'› ‹swap [(a,c)] t3'›]
      equ-abst-ab(1) by simp

    with 1(1,2) equ-abst-ab(2)
    show ?thesis using equ.equ-abst-ab[OF False a-fresh-t3] by simp
  qed
 next
  case (equ-abst-aa t2' t3' b)
  from this(3) 1(1)
  have i: nabla ⊢ swap [(a,b)] t2' ≈ swap [(a,b)] t3'
    using equ-equivariance[of nabla t2' t3' ‹[(a,b)]›] by simp

  have nabla ⊢ b ♯ swap [(a,b)] t2'
    using 1(1,3,4,5) equ-abst-aa(1) fresh-swap-eqvt[of nabla a t2' ‹[(a,b)]›]
    by auto
  then have ii: nabla ⊢ b ♯ swap [(a,b)] t3'
    using l3-jud[OF i, of b] by simp
  have nabla ⊢ swapas [(a,b)] b ♯ t3'
    using fresh-swap-left[OF ii] by simp
  then have a-fresh-t3: nabla ⊢ a ♯ t3'
    using 1(3,4) equ-abst-aa(1) by simp

  have is-equ: nabla ⊢ t1' ≈ swap [(a,b)] t3'
    using 1 i IH-usable[OF depth1, of ‹swap [(a,b)] t2'› ‹swap [(a,b)] t3'›]
    equ-abst-aa(1) by auto

  from a-fresh-t3 is-equ show ?thesis
    using 1 equ-abst-aa(1,2) equ.equ-abst-ab by simp
 qed (auto simp add: 1 t12)
next
 case (2 nabla t1' t2' a)
 have depth1: depth t1' < depth t1
  using depth.simps(4) 2(2) by simp
  from t23 show ?thesis
  proof(cases rule: equ.cases)
    case (equ-abst-ab a c t3' t2')
    have nabla ⊢ t1' ≈ swap [(a,c)] t3'
      using 2(1,2,3,4) IH-usable[OF depth1, of ‹t2'› ‹swap [(a,c)] t3'›]
     equ-abst-ab(1,4,5) by simp
    then show ?thesis
      using equ.equ-abst-ab[OF equ-abst-ab(3)]
        2(1,2,3) equ-abst-ab(1,2,4) by auto
  next
    case (equ-abst-aa t2' t3' a)
    then show ?thesis
      using 1 2(1,2,3,4) by auto
  qed (auto simp add: 2)
next
 case (3 nabla)
```

```
   from t23 show ?thesis
   proof(cases rule: equ.cases)
     case equ-unit
     then show ?thesis using t12 by auto
   qed (auto simp add: 3)
next
  case (4 a b nabla)
   from t23 show ?thesis
   proof(cases rule: equ.cases)
     case (equ-atom a b)
     then show ?thesis
       using t12 by auto
   qed (auto simp add: 4)
next
  case (5 pi1 pi2 X nabla)
   from t23 show ?thesis
   proof(cases rule: equ.cases)
     case (equ-susp pi1 pi2 X)
     then show ?thesis
       using ds-trans 5 by blast
   qed (auto simp add: 5)
next
  case (6 nabla t1' t2' s1' s2')
   from t23 show ?thesis
   proof(cases rule: equ.cases)
     case (equ-paar t2' t3' s2' s3')
     have depth1: depth t1' < depth t1
       using 6(2) by simp
    have depths1: depth s1' < depth t1
      using 6(2) by simp
    have a: nabla ⊢ t1' ≈ t2'
      using 6(3,4) equ-paar by auto
    have b: nabla ⊢ s1' ≈ s2'
      using 6 equ-paar by auto
    from a have t1-equal-t3: nabla ⊢ t1' ≈ t3'
      using IH-usable[of t1' t2' t3'] equ-paar(3) 6(1) depth1
      by simp
    from b have s1-equal-s3: nabla ⊢ s1' ≈ s3'
      using IH-usable[of s1' s2' s3'] equ-paar(4) 6(1) depths1
      by simp
    from t1-equal-t3 s1-equal-s3 show ?thesis
      using equ.equ-paar[of nabla t1' t3' s1' s3'] 6(1,2) equ-paar(2)
      by simp
   qed (auto simp add: 6)
next
  case (7 nabla t1 t2 F)
   from t23 show ?thesis
  proof(cases rule: equ.cases)
    case (equ-func t1 t2 F)
```

```
      then show ?thesis
        using 1 7(1,2,3,4) by auto
    qed (auto simp add: 7)
  qed
 qed
 then show ?case
  using IH(2,3) by blast
qed



lemma pi-right-equ-help:
 assumes (n = depth t)
 shows nabla ⊢ t ≈ swap pi t ⟹ ∀ a ∈ ds [] pi. nabla ⊢ a ♯ t
 using assms
proof(induction n arbitrary: t pi rule: nat-less-induct)
 case (1 n)
 note IH = this
 then show ?case
  proof(cases rule: equ.cases[OF ‹nabla ⊢ t ≈ swap pi t›])
   case (1 b c nabla t2 t1)
    have depth1: depth t1 < n
     using 1 depth.simps(4) IH by simp
    have swap-pi-t1-is-t2: swap pi t1 = t2
     using 1(2,3) by auto
    have swap-pi-b-is-c: swapas pi b = c
     using 1(2,3) by auto
    with swap-pi-t1-is-t2 have nabla ⊢ t1 ≈ swap [(b, swapas pi b)] (swap pi
t1)
     using 1(1,6) by simp
    then have nabla ⊢ t1 ≈ swap ([(b, swapas pi b)] @ pi) t1
     using swap-append[of ‹[(b, swapas pi b)]› pi t1] by simp
    with depth1 have ∀ a ∈ ds [] ((b, swapas pi b)#pi). nabla ⊢ a ♯ t1
     using 1.IH 1 by auto
    then have ds-minus-bc: ∀ a ∈ ds [] pi − {b, c}. nabla ⊢ a ♯ t1
     using ds-equality swap-pi-b-is-c by auto
    have c-fresh-t1: nabla ⊢ c ♯ t1
     using 1 equ-symm l3-jud fresh-swap-eqvt Equ-elims equ-abst-ab by meson
    with ds-minus-bc have ∀ a ∈ ds [] pi − {b}. nabla ⊢ a ♯ t1
     by auto
    then have ds-minus-b: ∀ a ∈ ds [] pi − {b}. nabla ⊢ a ♯ Abst b t1
     using fresh-abst-ab by blast
    have b-fresh-abst-t1: nabla ⊢ b ♯ Abst b t1
     using fresh-abst-aa by simp
    have ∀ a ∈ ds [] pi. nabla ⊢ a ♯ Abst b t1
    proof(rule, goal-cases)
     case (1 a)
     then show ?case
      using b-fresh-abst-t1 ds-minus-b
```

```
      by (cases a=b, auto)
    qed
    with 1 show ?thesis
      by auto
next
  case (2 nabla t1 t2 b)
  have depth t1: depth t1 < n
    using 2 depth.simps(4) IH by simp
  have swap-pi-t1-is-t2: swap pi t1 = t2
    using 2(2,3) by auto
  from swap-pi-t1-is-t2 depth t1
  have ∀ a ∈ ds [] pi. nabla ⊢ a ♯ t1
    using 1.IH 2(1,4) by auto
  then show ?thesis
    using 2(1,2,3) elem-ds by fastforce
next
  case (3 nabla)
  then show ?thesis
    by blast
next
  case (4 a b nabla)
  then show ?thesis
    using elem-ds by force
next
  case (5 pi1 pi2 X nabla)
  have swap pi t = Susp (pi@pi1) X
    using 5(2) by auto
  also have ... = Susp pi2 X
    using 5(3) calculation by simp
  then have pi@pi1 = pi2 by simp
  hence ∀ c∈ds pi1 (pi@pi1). (c, X) ∈ nabla
    using 5(4) by simp
  then show ?thesis
    using 5(1,2) fresh-susp ds-cancel-pi-right[of - - [] -]
    by simp
next
  case (6 nabla t1 t2 s1 s2)
  have depth t1: depth t1 < n
    using 6 depth.simps(5) IH
    by simp
  have depths1: depth s1 < n
    using 6 depth.simps(5) IH
    by simp
  from depth t1 depths1 show ?thesis
    using 1.IH 6 by auto
next
  case (7 nabla t1 t2 f)
  have depth t1: depth t1 < n
    using 7(2) depth.simps(5) IH
```

```
      by auto
    have nabla ⊢ t1 ≈ swap pi t1
      using 7 by simp
    then have ∀ a ∈ ds [] pi. nabla ⊢ a ♯ t1
      using 7(1) IH depth t1 by simp
    then show ?thesis
      using 7(1,2) fresh-func[of nabla - t1 f]
      by simp
  qed
qed


end
theory Equ

imports PreEqu

begin

lemma equ-refl:
  nabla ⊢ t ≈ t
  by(induct t, auto simp add: ds-def)


lemma equ-dec-pi:
  nabla ⊢ swap pi t1 ≈ swap pi t2 ⟹ nabla ⊢ t1 ≈ t2
proof−
  have i: nabla ⊢ swap (rev pi) (swap pi t1) ≈ t1
    nabla ⊢ swap (rev pi) (swap pi t2) ≈ t2
    using rev-pi-pi-equ by auto
  assume nabla ⊢ swap pi t1 ≈ swap pi t2
  then have nabla ⊢ swap (rev pi) (swap pi t1) ≈ swap (rev pi) (swap pi t2)
    using equ-equivariance by simp
  then show ?thesis using i equ-symm equ-trans by meson
qed


lemma equ-involutive-left:
  nabla ⊢ swap (rev pi) (swap pi t1) ≈ t2 = nabla ⊢ t1 ≈ t2
proof(auto)
  have i: nabla ⊢ t1 ≈ swap (rev pi) (swap pi t1)
    using rev-pi-pi-equ equ-symm by blast
  show nabla ⊢ swap (rev pi) (swap pi t1) ≈ t2 ⟹ nabla ⊢ t1 ≈ t2
    using i equ-trans by blast
  show nabla ⊢ t1 ≈ t2 ⟹ nabla ⊢ swap (rev pi) (swap pi t1) ≈ t2
    using i equ-trans equ-symm by blast
qed
```

lemma *equ-pi-to-left*:
nabla ⊢ *swap* (*rev pi*) *t1* ≈ *t2* = *nabla* ⊢ *t1* ≈ *swap pi t2*
proof

{assume *i*: *nabla* ⊢ *swap* (*rev pi*) *t1* ≈ *t2*
have *nabla* ⊢ *swap pi* (*swap* (*rev pi*) *t1*) ≈ *swap pi t2*
using *equ-equivariance*[*OF i*, *of pi*] **by** *simp*
then show *nabla* ⊢ *t1* ≈ *swap pi t2*
using *equ-involutive-left*[*of nabla* ‹*rev pi*› *t1* ‹*swap pi t2*›] *rev-rev-ident*[*of pi*]
by *simp***}**

{assume *i*: *nabla* ⊢ *t1* ≈ *swap pi t2*
have *ii*: *nabla* ⊢ *swap* (*rev pi*) *t1* ≈ *swap* (*rev pi*) (*swap pi t2*)
using *equ-equivariance*[*OF i*, *of* ‹*rev pi*›] **by** *simp*
then have *iii*: *nabla* ⊢ *swap* (*rev pi*) (*swap pi t2*) ≈ *swap* (*rev pi*) *t1*
using *equ-symm*[*OF ii*] **by** *simp*
then have *iv*: *nabla* ⊢ *t2* ≈ *swap* (*rev pi*) *t1*
using *equ-involutive-left*[*of nabla pi t2* ‹*swap* (*rev pi*) *t1*›] **by** *simp*
then show *nabla* ⊢ *swap* (*rev pi*) *t1* ≈ *t2*
using *equ-symm*[*OF iv*] **by** *simp***}**

qed

lemma *equ-pi-to-right*:
nabla⊢*t1* ≈ *swap* (*rev pi*) *t2* = *nabla*⊢*swap pi t1*≈*t2*
proof
{assume *i*: *nabla* ⊢ *t1* ≈ *swap* (*rev pi*) *t2*
then show *nabla* ⊢ *swap pi t1* ≈ *t2*
using *equ-involutive-left equ-dec-pi* **by** *blast***}**
{assume *ii*: *nabla* ⊢ *swap pi t1* ≈ *t2*
then show *nabla* ⊢ *t1* ≈ *swap* (*rev pi*) *t2*
using *equ-involutive-left equ-equivariance* **by** *blast***}**
qed

lemma *equ-involutive-right*:
nabla ⊢ *t1* ≈ *swap* (*rev pi*) (*swap pi t2*) = *nabla* ⊢ *t1* ≈ *t2*
using *equ-dec-pi*[*of nabla pi t1 t2*] *equ-equivariance*[*of nabla t1 t2 pi*]
equ-pi-to-right[*of nabla t1 pi* ‹*swap pi t2*›]
by *auto*

lemma *equ-pi1-pi2-add*:
(∀ *a*∈ *ds pi1 pi2*. *nabla*⊢*a*♯*t*) ⟹ (*nabla*⊢*swap pi1 t* ≈ *swap pi2 t*)
proof−
assume (∀ *a*∈ *ds pi1 pi2*. *nabla*⊢*a*♯*t*)
hence *nabla* ⊢ *t* ≈ *swap* (*rev pi1* @ *pi2*) *t*
using *ds-rev equ-pi-right* **by** *simp*
hence *nabla* ⊢ *t* ≈ *swap* (*rev pi1*) (*swap pi2 t*)

using *swap-append* **by** *auto*
then show *nabla*⊢*swap pi1 t* ≈ *swap pi2 t*
using *equ-pi-to-right* **by** *simp*
qed

lemma *pi-right-equ*: (*nabla* ⊢ *t* ≈ *swap pi t*) ⟹ (∀ *a*∈ *ds* [] *pi*. *nabla* ⊢ *a* ♯ *t*)
using *pi-right-equ-help* **by** *blast*

lemma *equ-pi1-pi2-dec*:
(*nabla* ⊢ *swap pi1 t* ≈ *swap pi2 t*) ⟹ (∀ *a* ∈ *ds pi1 pi2*. *nabla*⊢*a* ♯ *t*)
proof−
assume *assm*: *nabla* ⊢ *swap pi1 t* ≈ *swap pi2 t*
then have *nabla* ⊢ *t* ≈ *swap* ((*rev pi1*) @ *pi2*) *t*
using *equ-pi-to-right swap-append* **by** *simp*
then show ∀ *a* ∈ *ds pi1 pi2*. *nabla*⊢*a* ♯ *t*
using *pi-right-equ ds-rev*[*of pi1 pi2*] **by** *simp*
qed

lemma *equ-weak*:
nabla1 ⊢ *t1* ≈ *t2* ⟹ (*nabla1* ∪ *nabla2*) ⊢ *t1* ≈ *t2*
by(*erule equ.induct*, *auto simp add*: *fresh-weak*)

lemma *psub-trm-not-equ*:
∀ *t2* ∈ *psub-trms t1*. (¬(∃ *pi*. (*nabla* ⊢ *t1* ≈ *swap pi t2*)))
proof
fix *t2*
assume *i*: *t2* ∈ *psub-trms t1*
show ¬ (∃ *pi*. *nabla* ⊢ *t1* ≈ *swap pi t2*)
proof
assume ∃ *pi*. *nabla* ⊢ *t1* ≈ *swap pi t2*
then obtain *pi* **where** *H*:
nabla ⊢ *t1* ≈ *swap pi t2* **by** *blast*

from *equ-depth*[*OF H*]
have *depth t1* = *depth* (*swap pi t2*)
by *simp*
hence *depth t1* = *depth t2*
by *simp*
moreover have *depth t2* < *depth t1*
using *i depth-psub-trms*
by *simp*

ultimately show *False* **by** *auto*
qed
qed

end
theory *Substs*

imports *Equ*

begin

type-synonym *substs* = (*string* × *trm*) *list*

fun *look-up* :: *string* ⇒ *substs* ⇒ *trm* **where**
look-up X [] = *Susp* [] *X* |
look-up X (*x*#*xs*) = (*if* (*X* = *fst x*) *then* (*snd x*) *else look-up X xs*)

fun *subst* :: *substs* ⇒ *trm* ⇒ *trm* **where**
subst-unit: *subst s* (*Unit*) = *Unit* |
subst-susp: *subst s* (*Susp pi X*) = *swap pi* (*look-up X s*) |
subst-atom: *subst s* (*Atom a*) = *Atom a* |
subst-abst: *subst s* (*Abst a t*) = *Abst a* (*subst s t*) |
subst-paar: *subst s* (*Paar t1 t2*) = *Paar* (*subst s t1*) (*subst s t2*) |
subst-func: *subst s* (*Func F t*) = *Func F* (*subst s t*)

declare *subst-susp* [*simp del*]

fun *alist-rec* :: *substs* ⇒ *substs* ⇒ (*string*⇒*trm*⇒*substs*⇒*substs*⇒*substs*) ⇒ *substs*
where
alist-rec [] *c d* = *c* |
alist-rec (*p* # *al*) *c d* = *d* (*fst p*) (*snd p*) *al* (*alist-rec al c d*)

definition *comp* :: *substs* ⇒ *substs* ⇒ *substs* (**infixl** ‹•› *81*) **where**
s1 • *s2* ≡ *alist-rec s2 s1* (λ *x y xs g*. (*x*,*subst s1 y*) # *g*)

definition *domn* :: (*trm* ⇒ *trm*) ⇒ *string set* **where**
domn s ≡ {*X*. (*s* (*Susp* [] *X*)) ≠ (*Susp* [] *X*)}

definition *ext-subst* :: *fresh-envs* ⇒ (*trm* ⇒ *trm*) ⇒ *fresh-envs* ⇒ *bool* (- ⊨ - - [*80*,*80*,*80*] *80*) **where**

nabla1 $\models$ *s nabla2* $\equiv$ ($\forall$ (*a*,*X*)$\in$*nabla2*. *nabla1*$\vdash$*a*$\sharp$ *s* (*Susp* [] *X*))

definition *subst-equ* :: *fresh-envs* $\Rightarrow$ (*trm*$\Rightarrow$*trm*) $\Rightarrow$ (*trm*$\Rightarrow$*trm*) $\Rightarrow$ *bool* (- $\models$ - $\approx$ - [*80*,*80*,*80*] *80*)
where
nabla $\models$ *s1* $\approx$ *s2* $\equiv$ $\forall$ *X*$\in$(*domn s1*$\cup$*domn s2*). (*nabla* $\vdash$ *s1* (*Susp* [] *X*) $\approx$ *s2* (*Susp* [] *X*))

lemma *subst-equ-symm*:
assumes *nabla* $\models$ *s1* $\approx$ *s2*
shows *nabla* $\models$ *s2* $\approx$ *s1*
using *assms equ-symm subst-equ-def*[*of nabla s1 s2*]
subst-equ-def[*of nabla s2 s1*] *Un-commute*[*of* ‹*domn s1*› ‹*domn s2*›] **by** *blast*

lemma *subst-equ-refl*:
shows *nabla* $\models$ *s* $\approx$ *s*
using *equ-refl subst-equ-def Un-absorb* **by** *simp*

lemma *not-in-domn*: *X* $\notin$ (*domn s*)$\Longrightarrow$ (*s* (*Susp* [] *X*)) = (*Susp* [] *X*)
using *domn-def* **by** *simp*

lemma *subst-swap-comm*: *subst s* (*swap pi t*) = *swap pi* (*subst s t*)
using *swap-append subst-susp* **by** (*induct t*) *auto*

lemma *subst-not-occurs*:
¬(*occurs X t*) $\Longrightarrow$ *subst* [(*X*,*t2*)] *t* = *t*
proof−
have *i*: ¬(*occurs X t*) $\longrightarrow$ *subst* [(*X*,*t2*)] *t* = *t*
using *subst-susp* **by** (*induct t*) *auto*
assume ¬(*occurs X t*)
with *i* **show** *?thesis* **by** *simp*
qed

lemma *id-subst* [*simp*]: *subst* [] *t* = *t*
using *subst-susp* **by** (*induct t*) *auto*

lemma *subst-comp-id* [*simp*]: *subst* (*s* · []) = *subst s*
using *comp-def* **by** *simp*

lemma *id-comp-subst* [*simp*]: *subst* ([] · *s*) = *subst s*
proof(*rule ext*)
fix *t*
show *subst* ([] · *s*) *t* = *subst s t*
proof(*induction t arbitrary*: *s*)
case (*Susp pi X*)
then show *?case* **using** *comp-def subst-susp*

```
    proof (induction s)
      case Nil
      then show ?case
        using comp-def subst-susp by simp
    next
      case (Cons a s)
      then show ?case
        using comp-def subst-susp by simp
    qed
  qed (simp-all)
qed


lemma subst-comp-expand: subst (s1 • s2) t = subst s1 (subst s2 t)
proof(induct t)
  case (Susp x1 x2)
  then show ?case
   proof(induct s2)
     case Nil
     then show ?case using comp-def subst-susp subst-swap-comm by simp
   next
    case (Cons a s2)
    then show ?case using comp-def subst-susp subst-swap-comm by simp
  qed
qed (simp-all)



lemma subst-assoc: subst (s1 • (s2 • s3))=subst ((s1 • s2) • s3)
proof(rule ext)
  fix t
  show subst (s1 • (s2 • s3)) t = subst (s1 • s2 • s3) t
  proof(induct t)
    case (Susp pi X)
    then show ?case using subst-comp-expand by simp
  qed (simp-all)
qed


lemma fresh-subst:
  assumes nabla1 ⊢ a ♯ t
    and nabla2 ⊨ (subst s) nabla1
  shows nabla2 ⊢ a ♯ subst s t
  using assms
proof(induction rule: fresh.induct)
  case (fresh-susp pi a X nabla)
  then show ?case
    using ext-subst-def subst-susp fresh-swap-right
    by auto
```

qed (*auto*)

lemma *equ-subst*:
assumes *nabla1*⊢ *t1* ≈*t2*
and *nabla2* ⊨ (*subst s*) *nabla1*
shows *nabla2* ⊢(*subst s t1*)≈(*subst s t2*)
using *assms*
proof(*induction rule*: *equ.induct*)
case (*equ-abst-ab a b nabla t2 t1*)
then show *?case*
using *subst-swap-comm fresh-subst equ.equ-abst-ab* **by** *simp*
next
case (*equ-susp pi1 pi2 X nabla*)
then show *?case*
using *subst-susp equ-pi1-pi2-add ext-subst-def subst-susp*
by *fastforce*
qed (*auto*)

lemma *unif-1*:
assumes *nabla* ⊢ *subst s* (*Susp pi X*) ≈ *subst s t*
shows *nabla* ⊨ *subst* (*s* ∙ [(*X*, *swap* (*rev pi*) *t*)]) ≈ *subst s*
using *assms*
apply(*simp only*: *subst-equ-def*)
proof
fix *Xa*
assume *hyps*: *nabla* ⊢ *subst s* (*Susp pi X*) ≈ *subst s t*
Xa ∈ *domn* (*subst* (*s* ∙ [(*X*, *swap* (*rev pi*) *t*)])) ∪ *domn* (*subst s*)
show *nabla* ⊢ *subst* (*s* ∙ [(*X*, *swap* (*rev pi*) *t*)]) (*Susp* [] *Xa*) ≈ *subst s* (*Susp* [] *Xa*)
proof−
have *one*:
nabla ⊢ *subst* (*s* ∙ [(*X*, *swap* (*rev pi*) *t*)]) (*Susp pi Xa*) ≈
subst s (*subst* [(*X*, *swap* (*rev pi*) *t*)] (*Susp pi Xa*))
using *subst-comp-expand equ-refl* **by** *simp*
have *two*:
nabla ⊢ *subst s* (*subst* [(*X*, *swap* (*rev pi*) *t*)] (*Susp pi Xa*)) ≈
subst s (*swap pi* (*look-up Xa* [(*X*, *swap* (*rev pi*) *t*)]))
using *equ-refl subst-susp*[*of* ‹[(*X*, *swap* (*rev pi*)*t*)]› *pi Xa*]
by *simp*
have *nabla* ⊢ *subst* (*s* ∙ [(*X*, *swap* (*rev pi*) *t*)]) (*Susp pi Xa*) ≈
subst s (*swap pi* (*look-up Xa* [(*X*, *swap* (*rev pi*) *t*)]))
using *equ-trans*[*OF one two*] **by** *simp*
hence *nabla* ⊢ *swap pi* (*look-up Xa* (*s* ∙ [(*X*, *swap* (*rev pi*) *t*)])) ≈
subst s (*swap pi* (*look-up Xa* [(*X*, *swap* (*rev pi*) *t*)]))
using *subst-susp*[*of* ‹(*s* ∙ [(*X*, *swap* (*rev pi*) *t*)])›] **by** *simp*
hence *swap-pi-outside*: *nabla* ⊢ *swap pi* (*look-up Xa* (*s* ∙ [(*X*, *swap* (*rev pi*) *t*)])) ≈
swap pi (*subst s* (*look-up Xa* [(*X*, *swap* (*rev pi*) *t*)]))

```
    using subst-swap-comm by simp
  hence nabla ⊢ (look-up Xa (s • [(X, swap (rev pi) t)])) ≈
  (subst s (look-up Xa [(X, swap (rev pi) t)]))
    using equ-dec-pi[OF swap-pi-outside] by simp
  hence three:
  nabla ⊢ subst (s • [(X, swap (rev pi) t)]) (Susp [] Xa) ≈
  (subst s (look-up Xa [(X, swap (rev pi) t)]))
    using subst-susp by simp
  then show ?thesis
    proof(cases Xa = X)
      case True
     hence subst s (look-up Xa [(X, swap (rev pi) t)]) = subst s (swap (rev pi) t)
        by simp
      with three have
          nabla ⊢ subst (s • [(X, swap (rev pi) t)]) (Susp [] Xa) ≈ swap (rev pi)
(subst s t)
        using subst-swap-comm by auto
      hence i: nabla ⊢ swap pi (subst (s • [(X, swap (rev pi) t)]) (Susp [] Xa))
        ≈ subst s t using swap-inv-side by simp
      hence nabla ⊢ swap pi (subst (s • [(X, swap (rev pi) t)]) (Susp [] Xa))
        ≈ subst s (Susp pi Xa)
        using True equ-trans[OF i equ-symm[OF assms(1)]] by simp
      hence nabla ⊢ swap pi (subst (s • [(X, swap (rev pi) t)]) (Susp [] Xa))
        ≈ swap pi (look-up Xa s)
        using subst-susp by simp
      then show nabla ⊢ (subst (s • [(X, swap (rev pi) t)]) (Susp [] Xa))
        ≈ subst s (Susp [] Xa)
        using equ-dec-pi subst-susp by simp
    next
      case False
      hence subst s (look-up Xa [(X, swap (rev pi) t)]) = subst s (Susp [] Xa)
        by simp
      with three show ?thesis by auto
    qed
  qed
  qed

lemma subst-equ-a:
  assumes nabla ⊨ (subst s1) ≈ (subst s2)
    and nabla ⊢ (subst s2 t1) ≈ t2
  shows nabla ⊢ (subst s1 t1) ≈ t2
  using assms
proof(induct t1 arbitrary: t2)
  case (Abst a t1')
  then obtain t2' b
    where t2: t2 = Abst b t2'
    by(auto elim: equ.cases)
  with Abst(3) have
    i: nabla ⊢ Abst a (subst s2 t1') ≈ Abst b t2'
```

```
    using subst-abst by simp
  then show ?case
  proof(cases a=b)
    case True
    hence nabla ⊢ (subst s2 t1') ≈ t2'
      using Equ-elims(7)[OF i] by auto
    with Abst(1)[OF Abst(2), of t2']
    have nabla ⊢ subst s1 t1' ≈ t2'
      by simp
    hence nabla ⊢ Abst a (subst s1 t1') ≈ Abst b t2'
      using True equ-abst-aa by simp
    with t2 show ?thesis
      using subst-abst[of s1 a t1'] by simp
  next
    case False
    hence nabla ⊢ (subst s2 t1') ≈ swap [(a,b)] t2'
          and fresh: nabla ⊢ a ♯ t2'
      using Equ-elims(7)[OF i] by auto
     with Abst(1)[OF Abst(2), of ‹swap [(a,b)] t2'›]
    have nabla ⊢ subst s1 t1' ≈  swap [(a,b)] t2'
      by simp
    hence nabla ⊢ Abst a (subst s1 t1') ≈ Abst b t2'
      using equ-abst-ab[OF False fresh] by simp
    with t2 show ?thesis
      using subst-abst[of s1 a t1'] by simp
  qed
next
  case (Susp pi X)
  hence nabla ⊢ swap pi (look-up X s2) ≈ t2
    using subst-susp by simp
  hence nabla ⊢ (look-up X s2) ≈ swap (rev pi) t2
    using subst-susp swap-inv-side by simp
  hence i: nabla ⊢ subst s2 (Susp [] X) ≈ swap (rev pi) t2
    using subst-susp[of s2 ‹[]›] by simp

  with Susp(1) subst-equ-def[of nabla ‹subst s1› ‹subst s2›]
  have nabla ⊢ subst s1 (Susp [] X) ≈ subst s2 (Susp [] X)
    using UnCI equ-refl not-in-domn by metis

  with i have nabla ⊢ subst s1 (Susp [] X) ≈ swap (rev pi) t2
    using equ-trans by blast
  hence nabla ⊢ (look-up X s1) ≈ swap (rev pi) t2
    using subst-susp by simp
  hence nabla ⊢ swap pi (look-up X s1) ≈  t2
    using swap-inv-side by simp
    then show ?case
      using subst-susp by simp
next
  case (Paar t11 t12)
```

```
  then obtain t21 t22
    where t2: t2 = Paar t21 t22
    by(auto elim: equ.cases)
  with Paar(4)
  have paar-i: nabla ⊢ subst s2 t11 ≈ t21
    and paar-ii: nabla ⊢ subst s2 t12 ≈ t22
    by (auto elim: equ.cases)
  from t2 show ?case
   using equ-paar[OF Paar(1)[OF Paar(3) paar-i] Paar(2)[OF Paar(3) paar-ii]]
    subst-paar by simp
next
  case (Func f t1')
  then obtain t2'
    where t2: t2 = Func f t2'
    by(auto elim: equ.cases)
   with Func(3)
   have func: nabla ⊢ subst s2 t1' ≈ t2'
   by (auto elim: equ.cases)
  from t2 show ?case
    using equ-func[OF Func(1)[OF Func(2) func], of f] by simp
qed (simp-all)



lemma unif-2a:
  assumes nabla⊨subst s1≈subst s2
  and (nabla⊢subst s2 t1 ≈ subst s2 t2)
shows(nabla⊢subst s1 t1 ≈ subst s1 t2)
proof−
  have i: (nabla⊢subst s1 t1 ≈ subst s2 t2)
    using subst-equ-a[OF assms(1,2)] by simp
  show (nabla⊢subst s1 t1 ≈ subst s1 t2)
    using subst-equ-a[OF assms(1) equ-symm[OF i]] equ-symm by auto
qed

lemma unif-2b:
  assumes nabla ⊨ subst s1 ≈ subst s2
  and nabla ⊢ a ♯ subst s2 t
shows nabla ⊢ a ♯ subst s1 t
  using assms
proof(induct t)
  case (Abst b t')
  then show ?case
  proof(cases a=b)
    case True
    then show ?thesis
      using fresh-abst-aa by simp
  next
    case False
```

with *Abst* **show** *?thesis*
using *Fresh-elims*(*1*) *Substs.subst-abst fresh-abst-ab*
by *metis*
qed
next
case (*Susp pi X*)
then show *?case*
using *equ-refl equ-symm l3-jud subst-equ-a* **by** *meson*
next
case (*Paar t1 t2*)
then show *?case*
using *Fresh-elims*(*5*) *subst-paar fresh-paar*
by *metis*
next
case (*Func f t'*)
then show *?case*
using *Fresh-elims*(*6*) *subst-func fresh-func*
by *metis*
qed (*simp-all*)

lemma *subst-equ-to-trm*: *nabla* ⊨ *subst s1* ≈ *subst s2* ⟹ *nabla*⊢*subst s1 t* ≈ *subst s2 t*
using *equ-refl subst-equ-a* **by** *simp*

lemma *subst-cancel-right*:
⟦*nabla*⊨(*subst s1*)≈(*subst s2*)⟧⟹*nabla*⊨(*subst* (*s1*•*s*))≈(*subst* (*s2*•*s*))
using *subst-comp-expand subst-equ-def subst-equ-to-trm*
by *simp*

lemma *subst-trans*: ⟦*nabla*⊨*subst s1* ≈ *subst s2*; *nabla* ⊨ *subst s2* ≈ *subst s3*⟧⟹*nabla*⊨*subst s1*≈*subst s3*
using *equ-trans subst-equ-def subst-equ-to-trm* **by** *meson*

lemma *occurs-sub-trm-equ*:
occurs X t1 ⟹ (∃ *t2* ∈ *sub-trms* (*subst s t1*). (∃*pi*.(*nabla*⊢*subst s* (*Susp pi1 X*)≈(*swap pi t2*))))
proof(*induct t1*)
case (*Susp pi' X*)
then show *?case*
using *equ-pi-to-left equ-involutive-left*
equ-refl subst-susp swap-append t-sub-trms-t
occurs.simps(*3*) **by** *metis*
next
case (*Paar t11 t12*)
then show *?case*

using *Substs.subst-paar Un-iff occurs.simps*(*5*) *psub-sub-trms*
psub-trms.simps(*4*) **by** (*metis*)
qed(*auto*)

lemma *ext-subst-strong*:
assumes *nabla1* $\models$ (*subst s*) (*nabla2* $\cup$ *nabla3*)
shows *nabla1* $\models$ (*subst s*) (*nabla2*) **and** *nabla1* $\models$ (*subst s*) (*nabla3*)
using *assms*
unfolding *ext-subst-def* **by** *auto*

lemma *ext-subst-id*:
shows *nabla* $\models$ (*subst* []) *nabla*
unfolding *ext-subst-def id-subst* **by** *auto*

end
theory *Mgu*

imports *Substs*

begin

syntax *equ-prob* :: *trm* $\Rightarrow$ *trm* $\Rightarrow$ (*trm* $\times$ *trm*) (**infix** $\approx$*?* *81*)
fresh-prob :: *string* $\Rightarrow$ *trm* $\Rightarrow$ (*string* $\times$ *trm*) (**infix** $\sharp$*?* *81*)
translations
t1 $\approx$*?* *t2* $\rightharpoonup$ (*t1*, *t2*)
a $\sharp$*?* *t* $\rightharpoonup$ (*a*,*t*)

type-synonym *problem-type* = ((*trm* $\times$ *trm*) *list*) $\times$ ((*string* $\times$ *trm*) *list*)

type-synonym *unifier-type* = *fresh-envs* $\times$ *substs*

definition *U*:: *problem-type* $\Rightarrow$ (*unifier-type set*)
where *all-solutions-def*:
U P $\equiv$ {(*nabla*, *s*).
($\forall$ (*t1*,*t2*) $\in$ *set* (*fst P*). *nabla* $\vdash$ *subst s t1* $\approx$ *subst s t2*) $\wedge$
($\forall$ (*a*,*t*) $\in$ *set* (*snd P*). *nabla* $\vdash$ *a* $\sharp$ *subst s t*)}

type-synonym *eprobs* = ((*trm* $\times$ *trm*) *list*)
type-synonym *fprobs* = ((*string* $\times$ *trm*) *list*)
type-synonym *probs* = *eprobs* $\times$ *fprobs*

fun *vars-fprobs*:: ((*string* × *trm*) *list*) ⇒ (*string set*)
where
vars-fprobs [] = {} |
vars-fprobs (*x*#*xs*) = (*vars-trm* (*snd x*))∪(*vars-fprobs xs*)

fun *vars-eprobs* :: ((*trm* × *trm*) *list*) ⇒ (*string set*)
where
vars-eprobs [] = {} |
vars-eprobs (*x*#*xs*) = (*vars-trm* (*snd x*))∪(*vars-trm* (*fst x*))∪(*vars-eprobs xs*)

definition *vars-probs* :: *problem-type* ⇒ *nat*
where *vars-probs P* ≡ *card*((*vars-fprobs* (*snd P*))∪(*vars-eprobs* (*fst P*)))

definition *mgu* :: *problem-type* ⇒ *unifier-type* ⇒ *bool*
where *mgu P unif* ≡
∀ (*nabla*,*s1*)∈ *U P*. (∃ *s2*. (*nabla* ⊨ (*subst s2*) (*fst unif*)) ∧
(*nabla* ⊨ *subst* (*s2* ∘(*snd unif*)) ≈ *subst s1*))

definition *idem* :: *unifier-type* ⇒ *bool*
where *idem unif* ≡ (*fst unif*)⊨ *subst* ((*snd unif*)∘(*snd unif*)) ≈ *subst* (*snd unif*)

definition *apply-subst-eprobs* :: *substs* ⇒ *eprobs* ⇒ *eprobs*
where *apply-subst-eprobs s P* ≡ *map* (λ(*t1*, *t2*). (*subst s t1* ≈*? subst s t2*)) *P*

definition *apply-subst-fprobs* :: *substs* ⇒ *fprobs* ⇒ *fprobs*
where *apply-subst-fprobs s P* ≡ *map* (λ(*a*, *t*). (*a* ♯*? subst s t*)) *P*

definition *apply-subst* :: *substs* ⇒ *problem-type* ⇒ *problem-type*
where *apply-subst s P* ≡ (*map* (λ(*t1*, *t2*). (*subst s t1* ≈*? subst s t2*)) (*fst P*),
map (λ(*a*, *t*). (*a* ♯*?* (*subst s t*))) (*snd P*))

inductive *s-red* :: *problem-type* ⇒ *substs* ⇒ *problem-type* ⇒ *bool* (- ⊢ - ⇝ - [*80*,*80*,*80*] *80*)

where
unit-sred[*intro*!]: $((Unit \approx ?\ Unit)\ \#\ xs,\ ys) \vdash [] \rightsquigarrow (xs,\ ys)$ |
paar-sred[*intro*!]: $((Paar\ t1\ t2 \approx ?\ Paar\ s1\ s2)\#xs,ys) \vdash [] \rightsquigarrow ((t1 \approx ?s1)\#(t2 \approx ?s2)\#xs,ys)$
|
func-sred[*intro*!]: $((Func\ F\ t1 \approx ?\ Func\ F\ t2)\#xs,ys) \vdash [] \rightsquigarrow ((t1 \approx ?t2)\#xs,ys)$ |
abst-aa-sred[*intro*!]: $((Abst\ a\ t1 \approx ?\ Abst\ a\ t2)\#xs,ys) \vdash [] \rightsquigarrow ((t1 \approx ?t2)\#xs,ys)$ |
abst-ab-sred[*intro*!]: $a \neq b \Longrightarrow$
$((Abst\ a\ t1 \approx ?Abst\ b\ t2)\#xs,ys) \vdash [] \rightsquigarrow ((t1 \approx ?swap\ [(a,b)]\ t2)\#xs,(a\sharp ?t2)\#ys)$ |
atom-sred[*intro*!]: $((Atom\ a \approx ?Atom\ a)\#xs,ys) \vdash [] \rightsquigarrow (xs,ys)$ |
susp-sred[*intro*!]: $((Susp\ pi1\ X \approx ?Susp\ pi2\ X)\#xs,ys)$
$\vdash [] \rightsquigarrow (xs,(map\ (\lambda a.\ a\sharp ?\ Susp\ []\ X)\ (ds\text{-}list\ pi1\ pi2))@ys)$ |
var-1-sred[*intro*!]: $\neg(occurs\ X\ t) \Longrightarrow ((Susp\ pi\ X \approx ?t)\#xs,ys)$
$\vdash [(X,swap\ (rev\ pi)\ t)] \rightsquigarrow apply\text{-}subst\ [(X,swap\ (rev\ pi)\ t)](xs,ys)$ |
var-2-sred[*intro*!]: $\neg(occurs\ X\ t) \Longrightarrow ((t \approx ?Susp\ pi\ X)\#xs,ys)$
$\vdash [(X,swap\ (rev\ pi)\ t)] \rightsquigarrow apply\text{-}subst\ [(X,swap\ (rev\ pi)\ t)](xs,ys)$

inductive-cases *s-red-elims*:
$((Unit \approx ?\ Unit)\ \#\ xs,\ ys) \vdash [] \rightsquigarrow (xs,\ ys)$
$((Paar\ t1\ t2 \approx ?\ Paar\ s1\ s2)\#xs,ys) \vdash [] \rightsquigarrow ((t1 \approx ?s1)\#(t2 \approx ?s2)\#xs,ys)$
$((Func\ F\ t1 \approx ?\ Func\ F\ t2)\#xs,ys) \vdash [] \rightsquigarrow ((t1 \approx ?t2)\#xs,ys)$
$((Abst\ a\ t1 \approx ?\ Abst\ a\ t2)\#xs,ys) \vdash [] \rightsquigarrow ((t1 \approx ?t2)\#xs,ys)$
$((Abst\ a\ t1 \approx ?Abst\ b\ t2)\#xs,ys) \vdash [] \rightsquigarrow ((t1 \approx ?swap\ [(a,b)]\ t2)\#xs,(a\sharp ?t2)\#ys)$
$((Atom\ a \approx ?Atom\ a)\#xs,ys) \vdash [] \rightsquigarrow (xs,ys)$
$((Susp\ pi1\ X \approx ?Susp\ pi2\ X)\#xs,ys) \vdash [] \rightsquigarrow (xs,(map\ (\lambda a.\ a\sharp ?\ Susp\ []\ X)\ (ds\text{-}list\ pi1\ pi2))@ys)$
$((Susp\ pi\ X \approx ?t)\#xs,ys) \vdash [(X,swap\ (rev\ pi)\ t)] \rightsquigarrow apply\text{-}subst\ [(X,swap\ (rev\ pi)\ t)](xs,ys)$
$((t \approx ?Susp\ pi\ X)\#xs,ys) \vdash [(X,swap\ (rev\ pi)\ t)] \rightsquigarrow apply\text{-}subst\ [(X,swap\ (rev\ pi)\ t)](xs,ys)$

lemma *solution-weak*:
assumes $(nabla1,\ s) \in U\ P$
shows $(nabla1 \cup nabla3,\ s) \in U\ P$
using *assms*
unfolding *all-solutions-def* **using** *fresh-weak equ-weak* **by** *auto*

lemma *solution-comp-id*:
shows $((nabla,\ s) \in U\ P) = ((nabla,\ s \cdot []) \in U\ P)$ **and**
$((nabla,\ s) \in U\ P) = ((nabla,\ [] \cdot s) \in U\ P)$
unfolding *all-solutions-def* **by** *auto*

lemma *solutions-subst-assoc*:
$((nabla,\ s1 \cdot (s2 \cdot s3)) \in U\ P) = ((nabla,\ (s1 \cdot s2) \cdot s3) \in U\ P)$

unfolding *all-solutions-def* **using** *subst-assoc* **by** *simp*

inductive *c-red* :: *problem-type* ⇒ *fresh-envs* ⇒ *problem-type* ⇒ *bool* (- ⊢ - → - [*80*,*80*,*80*] *80*)
where
unit-cred[*intro*!]: ([],(*a* ♯? *Unit*)#*xs*) ⊢{}→ ([],*xs*) |
paar-cred[*intro*!]: ([],(*a* ♯? *Paar t1 t2*)#*xs*) ⊢{}→ ([],(*a*♯?*t1*)#(*a*♯?*t2*)#*xs*) |
func-cred[*intro*!]: ([],(*a* ♯? *Func F t*)#*xs*) ⊢{}→ ([],(*a*♯?*t*)#*xs*) |
abst-aa-cred[*intro*!]: ([],(*a* ♯? *Abst a t*)#*xs*) ⊢{}→ ([],*xs*) |
abst-ab-cred[*intro*!]: *a*≠*b* ⟹([],(*a* ♯? *Abst b t*)#*xs*) ⊢{}→ ([],(*a*♯?*t*)#*xs*) |
atom-cred[*intro*!]: *a*≠*b* ⟹([],(*a* ♯? *Atom b*)#*xs*) ⊢{}→ ([],*xs*) |
susp-cred[*intro*!]: ([],(*a* ♯? *Susp pi X*)#*xs*) ⊢ {((*swapas* (*rev pi*) *a*),*X*)}→ ([],*xs*)

inductive *red-plus* :: *problem-type* ⇒ *unifier-type* ⇒ *problem-type* ⇒ *bool* (- ⊨ - ⇒ - [*80*,*80*,*80*] *80*)
where
sred-single[*intro*!]: ⟦*P1*⊢*s1* ⇝ *P2*⟧ ⟹ *P1*⊨({},*s1*)⇒*P2* |
cred-single[*intro*!]: ⟦*P1*⊢*nabla1*→*P2*⟧ ⟹ *P1* ⊨ (*nabla1*,[]) ⇒ *P2* |
sred-step[*intro*!]: ⟦*P1*⊢*s1*⇝*P2*; *P2*⊨(*nabla2*,*s2*)⇒*P3*⟧ ⟹ *P1*⊨(*nabla2*,(*s2*•*s1*))⇒*P3* |
cred-step[*intro*!]: ⟦*P1*⊢*nabla1*→*P2*; *P2*⊨(*nabla2*,[])⇒*P3*⟧ ⟹ *P1*⊨(*nabla2*∪*nabla1*,[])⇒*P3*

lemma *mgu-idem*:
assumes (*nabla1*,*s1*) ∈ *U P*
and ∀(*nabla2*,*s2*) ∈ *U P*. *nabla2* ⊨(*subst s2*) *nabla1* ∧ *nabla2*⊨*subst*(*s2* • *s1*)≈*subst s2*
shows *mgu P* (*nabla1*,*s1*) ∧ *idem* (*nabla1*,*s1*)
using *assms mgu-def idem-def* **by** *auto*

lemma *problem-subst-comm*: ((*nabla*,*s2*) ∈ *U* (*apply-subst s1 P*)) = ((*nabla*,(*s2*•*s1*))∈*U P*)
using *all-solutions-def apply-subst-def subst-comp-expand* **by** *auto*

lemma *P1-to-P2-sred*:
assumes (*nabla1*,*s1*) ∈ *U P1* **and** *P1* ⊢*s2* ⇝ *P2*
shows (*nabla1*,*s1*) ∈ *U P2* ∧ (*nabla1*⊨*subst* (*s1*•*s2*)≈*subst s1*)
using *assms*
proof(*induction arbitrary*: *nabla1 s1 rule*: *s-red.induct*[*OF* ‹*P1* ⊢ *s2* ⇝ *P2*›])
case (*1 xs ys*)
then have *subst*: *nabla1* ⊨ *subst* (*s1* • []) ≈ *subst s1*

```
    using equ-refl subst-equ-def by simp
  from 1
  have eqs: ∀ (t1,t2) ∈ set xs. nabla1 ⊢ subst s1 t1 ≈ subst s1 t2
    and fresh: ∀ (a,t) ∈ set ys. nabla1 ⊢ a ♯ subst s1 t
    using all-solutions-def by simp-all
  with subst show ?case
    using all-solutions-def by auto
next
  case (2 t1 t2 t3 t4 xs ys)
  then have subst: nabla1 ⊨ subst (s1• []) ≈ subst s1
    using equ-refl subst-equ-def by simp

  from 2
  have fresh: ∀ (a,t) ∈ set ys. nabla1 ⊢ a ♯ subst s1 t
    using all-solutions-def by simp

  from 2
  have i: ∀ (t1,t2) ∈ set ((Paar t1 t2, Paar t3 t4) # xs). nabla1 ⊢ subst s1 t1
≈ subst s1 t2
    using all-solutions-def by simp
  hence nabla1 ⊢ subst s1 t1 ≈ subst s1 t3
   nabla1 ⊢ subst s1 t2 ≈ subst s1 t4
    using Equ-elims(4)[of nabla1 ‹(subst s1 t1)› ‹(subst s1 t2)› ‹(subst s1 t3)›
‹(subst s1 t4)›]
     subst-paar[of s1] by auto+
   with i have eqs:
   ∀ (t1,t2) ∈ set ((t1,t3)#(t2, t4) # xs). nabla1 ⊢ subst s1 t1 ≈ subst s1 t2
     by auto

   from fresh eqs subst
   show ?case using all-solutions-def by simp
next
  case (3 F t1 t2 xs ys)
  then have subst: nabla1 ⊨ subst (s1 • []) ≈ subst s1
    using equ-refl subst-equ-def by simp

  from 3
  have fresh: ∀ (a,t) ∈ set ys. nabla1 ⊢ a ♯ subst s1 t
    using all-solutions-def by simp

  from 3
  have i: ∀ (t1,t2) ∈ set ((Func F t1, Func F t2) # xs). nabla1 ⊢ subst s1 t1 ≈
subst s1 t2
    using all-solutions-def by simp
  hence nabla1 ⊢ subst s1 t1 ≈ subst s1 t2
    using Equ-elims(5)[of nabla1 F ‹subst s1 t1› ‹subst s1 t2›]
     subst-func by auto
  with i have eqs:
    ∀ (t1,t2) ∈ set ((t1, t2) # xs). nabla1 ⊢ subst s1 t1 ≈ subst s1 t2
```

```
    by auto

  from subst fresh eqs
  show ?case using all-solutions-def by simp
 next
  case (4 a t1 t2 xs ys)
  then have subst: nabla1 ⊨ subst (s1 · []) ≈ subst s1
    using equ-refl subst-equ-def by simp

  from 4
  have fresh: ∀ (a,t) ∈ set ys. nabla1 ⊢ a ♯ subst s1 t
    using all-solutions-def by simp

  from 4
  have i: ∀ (t1,t2) ∈ set ((Abst a t1, Abst a t2) # xs). nabla1 ⊢ subst s1 t1 ≈
subst s1 t2
    using all-solutions-def by simp
  hence nabla1 ⊢ subst s1 t1 ≈ subst s1 t2
    using Equ-elims(6)[of nabla1 a ‹subst s1 t1› ‹subst s1 t2›]
     subst-abst by auto
  with i have eqs:
    ∀ (t1,t2) ∈ set ((t1, t2) # xs). nabla1 ⊢ subst s1 t1 ≈ subst s1 t2
    by auto

  from subst fresh eqs
  show ?case using all-solutions-def by simp
 next
  case (5 a b t1 t2 xs ys)
  then have subst: nabla1 ⊨ subst (s1 · []) ≈ subst s1
    using equ-refl subst-equ-def by simp

  from 5
  have eqsP1: ∀ (t1,t2) ∈ set ((Abst a t1, Abst b t2) # xs). nabla1 ⊢ subst s1 t1
≈ subst s1 t2
    using all-solutions-def by simp
  hence i: nabla1 ⊢ Abst a (subst s1 t1) ≈ Abst b (subst s1 t2)
    using subst-abst[of s1] by simp

  from 5
  have ii: ∀ (a,t) ∈ set ys. nabla1 ⊢ a ♯ subst s1 t
    using all-solutions-def by auto
  from 5 i
  have nabla1 ⊢ a ♯ subst s1 t2
    using Equ-elims(7) by blast
  with ii
  have fresh: ∀ (a,t) ∈ set ((a,t2) # ys). nabla1 ⊢ a ♯ subst s1 t
    by simp

  from i 5
```

```
  have nabla1 ⊢ subst s1 t1 ≈ subst s1 (swap [(a, b)] t2)
    using 5 Equ-elims(7)[of nabla1 a ‹subst s1 t1› b ‹subst s1 t2›]
      subst-swap-comm by simp
  with eqsP1 have eqs:
   ∀ (t1,t2) ∈ set ((t1, swap [(a,b)] t2) # xs). nabla1 ⊢ subst s1 t1 ≈ subst s1 t2
    by auto

  from subst fresh eqs
  show ?case using all-solutions-def by simp
next
  case (6 a xs ys)
  then have subst: nabla1 ⊨ subst (s1 · []) ≈ subst s1
    using equ-refl subst-equ-def by simp

  from 6
  have fresh: ∀ (a,t) ∈ set ys. nabla1 ⊢ a ♯ subst s1 t
    using all-solutions-def by auto

  from 6
  have eqs: ∀ (t1,t2) ∈ set xs. nabla1 ⊢ subst s1 t1 ≈ subst s1 t2
    using all-solutions-def by auto

  from subst fresh eqs
  show ?case using all-solutions-def by simp
next
  case (7 pi1 X pi2 xs ys)
  then have subst: nabla1 ⊨ subst (s1 · []) ≈ subst s1
    using equ-refl subst-equ-def by simp

  from 7
  have i: ∀ (t1,t2) ∈ set ((Susp pi1 X, Susp pi2 X) # xs). nabla1 ⊢ subst s1 t1
≈ subst s1 t2
    ∀ (a,t) ∈ set ys. nabla1 ⊢ a ♯ subst s1 t
    using all-solutions-def by simp+


  from 7(1)
  have nabla1 ⊢ swap pi1 (look-up X s1) ≈ swap pi2 (look-up X s1)
    using all-solutions-def subst-susp by simp
  hence ∀ a∈ds pi1 pi2.  nabla1 ⊢ a ♯ (look-up X s1)
    using equ-pi1-pi2-dec by simp
  hence ∀ a∈ set (ds-list pi1 pi2). nabla1 ⊢ a ♯ subst s1 (Susp [] X)
    using subst-susp ds-list-equ-ds[of pi1 pi2] by simp
  hence ∀ (a,t) ∈ set (map (λa. a ♯? Susp [] X) (ds-list pi1 pi2)). nabla1 ⊢ a ♯
subst s1 t
   by (auto split: prod.splits)
  with 7 i(2)
  have fresh:
    ∀ (a,t) ∈ set (map (λa. (a, Susp [] X)) (ds-list pi1 pi2) @ ys). nabla1 ⊢ a ♯
```

subst s1 t
by *auto*

from *i(1)*
have *eqs*:
∀ *(t1,t2)* ∈ *set xs. nabla1* ⊢ *subst s1 t1* ≈ *subst s1 t2*
by *simp*

from *subst fresh eqs*
show *?case* **using** *all-solutions-def* **by** *simp*
next
case *(8 X t′ pi xs ys)*
hence *nabla1* ⊢ *subst s1* (*Susp pi X*) ≈ *subst s1 t′*
using *all-solutions-def*[*of* ‹((*Susp pi X*, *t′*) # *xs*, *ys*)›] **by** *auto*
hence *subst*: *nabla1* ⊨ *subst* (*s1* ∙ [(*X*, *swap* (*rev pi*) *t′*)]) ≈ *subst s1*
using *unif-1* **by** *simp*

from *8*
have *eqs-old*:
∀*(t1,t2)*∈*set xs. nabla1* ⊢ *subst s1 t1* ≈ *subst s1 t2*
and *fresh-old*:
∀*(a,t)*∈*set ys. nabla1* ⊢ *a* ♯ *subst s1 t*
using *all-solutions-def* **by** *auto*

from *eqs-old*
have *eqs*: ∀*(t1,t2)*∈*set xs.*
nabla1 ⊢ *subst* (*s1* ∙ [(*X*, *swap* (*rev pi*) *t′*)]) *t1* ≈ *subst* (*s1* ∙ [(*X*, *swap* (*rev pi*) *t′*)]) *t2*
using *unif-2a*[*OF subst*] **by** *auto*

from *fresh-old*
have *fresh*: ∀*(a,t)*∈*set ys. nabla1* ⊢ *a* ♯ *subst* (*s1* ∙ [(*X*, *swap* (*rev pi*) *t′*)]) *t*
using *unif-2b*[*OF subst*] **by** *auto*

from *eqs fresh*
have (*nabla1*, (*s1* ∙ [(*X*, *swap* (*rev pi*) *t′*)])) ∈ *U* (*xs*, *ys*)
using *all-solutions-def* **by** *simp*
hence *U*: (*nabla1*, *s1*) ∈ *U* (*apply-subst* [(*X*, *swap* (*rev pi*) *t′*)] (*xs*, *ys*))
using *problem-subst-comm* **by** *simp*

from *subst U*
show *?case* **by** *simp*
next
case *(9 X t′ pi xs ys)*
hence *nabla1* ⊢ *subst s1 t′* ≈ *subst s1* (*Susp pi X*)
using *all-solutions-def*[*of* ‹((*t′*, *Susp pi X*) # *xs*, *ys*)›] **by** *simp*
hence *subst*: *nabla1* ⊨ *subst* (*s1* ∙ [(*X*, *swap* (*rev pi*) *t′*)]) ≈ *subst s1*
using *unif-1*[*OF equ-symm*] **by** *blast*

from *9*
have *eqs-old*:
$\forall (t1,t2) \in set\ xs.\ nabla1 \vdash subst\ s1\ t1 \approx subst\ s1\ t2$
and *fresh-old*:
$\forall (a,t) \in set\ ys.\ nabla1 \vdash a \sharp subst\ s1\ t$
using *all-solutions-def* **by** *auto*

from *eqs-old*
have *eqs*: $\forall (t1,t2) \in set\ xs.$
$nabla1 \vdash subst\ (s1 \cdot [(X, swap\ (rev\ pi)\ t')])\ t1 \approx subst\ (s1 \cdot [(X, swap\ (rev\ pi)\ t')])\ t2$
using *unif-2a*[*OF subst*] **by** *auto*

from *fresh-old*
have *fresh*: $\forall (a,t) \in set\ ys.\ nabla1 \vdash a \sharp subst\ (s1 \cdot [(X, swap\ (rev\ pi)\ t')])\ t$
using *unif-2b*[*OF subst*] **by** *auto*

from *eqs fresh*
have $(nabla1, (s1 \cdot [(X, swap\ (rev\ pi)\ t')])) \in U\ (xs, ys)$
using *all-solutions-def* **by** *simp*
hence *U*: $(nabla1, s1) \in U\ (apply\text{-}subst\ [(X, swap\ (rev\ pi)\ t')]\ (xs, ys))$
using *problem-subst-comm* **by** *simp*

from *subst U*
show *?case* **by** *simp*
qed

lemma *P1-from-P2-sred*:
assumes $(nabla1,s1) \in U\ P2$ **and** $P1 \vdash s2 \rightsquigarrow P2$
shows $(nabla1, s1 \cdot s2) \in U\ P1$
using *assms*
proof(*induction arbitrary*: *nabla1 s1 rule*: *s-red.induct*[*OF* ‹*P1* ⊢ *s2* ⇝ *P2*›])
case (*1 xs ys*)
hence $\forall\ (t1,t2) \in set\ xs.\ nabla1 \vdash subst\ (s1 \cdot [])\ t1 \approx subst\ (s1 \cdot [])\ t2$
and *fresh*: $\forall\ (a,t) \in set\ ys.\ nabla1 \vdash a \sharp subst\ (s1 \cdot [])\ t$
using *all-solutions-def* **by** *simp-all*
hence *eqs*:
$\forall\ (t1,t2) \in set\ ((Unit, Unit)\ \#\ xs).\ nabla1 \vdash subst\ (s1 \cdot [])\ t1 \approx subst\ (s1 \cdot [])\ t2$
using *subst-unit*[*of* ‹(*s1* · [])›] *equ-refl*[*of nabla1 Unit*] **by** *auto*
from *fresh eqs* **show** *?case*
using *all-solutions-def* **by** *simp*
next
case (*2 t1 t2 t3 t4 xs ys*)
hence *eqs-old*: $\forall\ (t1,t2) \in set\ ((t1, t3)\ \#\ (t2, t4)\ \#\ xs).\ nabla1 \vdash subst\ (s1 \cdot [])\ t1 \approx subst\ (s1 \cdot [])\ t2$
and *fresh*: $\forall\ (a,t) \in set\ ys.\ nabla1 \vdash a \sharp subst\ (s1 \cdot [])\ t$

using *all-solutions-def* **by** *simp-all*
hence *nabla1* ⊢ *subst* (*s1* · []) *t1* ≈ *subst* (*s1* · []) *t3*
nabla1 ⊢ *subst* (*s1* · []) *t2* ≈ *subst* (*s1* · []) *t4* **by** *simp-all*
hence *nabla1* ⊢ *subst* (*s1* · []) (*Paar t1 t2*) ≈ *subst* (*s1* · []) (*Paar t3 t4*)
using *equ-paar* **by** *simp*
with *eqs-old* **have** *eqs*:
∀ (*t1*,*t2*) ∈ *set* ((*Paar t1 t2*, *Paar t3 t4*) # *xs*). *nabla1* ⊢ *subst* (*s1* · []) *t1* ≈ *subst* (*s1* · []) *t2*
by *auto*
from *fresh eqs* **show** *?case*
using *all-solutions-def* **by** *simp*
next
case (*3 f t1 t2 xs ys*)
hence *eqs-old*: ∀ (*t1*,*t2*) ∈ *set* ((*t1*, *t2*) # *xs*). *nabla1* ⊢ *subst* (*s1* · []) *t1* ≈ *subst* (*s1* · []) *t2*
and *fresh*: ∀ (*a*,*t*) ∈ *set ys*. *nabla1* ⊢ *a* ♯ *subst* (*s1* · []) *t*
using *all-solutions-def* **by** *simp-all*
hence *nabla1* ⊢ *subst* (*s1* · []) (*Func f t1*) ≈ *subst* (*s1* · []) (*Func f t2*)
using *equ-func* **by** *simp*
with *eqs-old* **have** *eqs*:
∀ (*t1*,*t2*) ∈ *set* ((*Func f t1*, *Func f t2*) # *xs*). *nabla1* ⊢ *subst* (*s1* · []) *t1* ≈ *subst* (*s1* · []) *t2*
by *auto*
from *fresh eqs* **show** *?case*
using *all-solutions-def* **by** *simp*
next
case (*4 a t1 t2 xs ys*)
hence *eqs-old*: ∀ (*t1*,*t2*) ∈ *set* ((*t1*, *t2*) # *xs*). *nabla1* ⊢ *subst* (*s1* · []) *t1* ≈ *subst* (*s1* · []) *t2*
and *fresh*: ∀ (*a*,*t*) ∈ *set ys*. *nabla1* ⊢ *a* ♯ *subst* (*s1* · []) *t*
using *all-solutions-def* **by** *simp-all*
hence *nabla1* ⊢ *subst* (*s1* · []) (*Abst a t1*) ≈ *subst* (*s1* · []) (*Abst a t2*)
using *equ-abst-aa* **by** *simp*
with *eqs-old* **have** *eqs*:
∀ (*t1*,*t2*) ∈ *set* ((*Abst a t1*, *Abst a t2*) # *xs*). *nabla1* ⊢ *subst* (*s1* · []) *t1* ≈ *subst* (*s1* · []) *t2*
by *auto*
from *fresh eqs* **show** *?case*
using *all-solutions-def* **by** *simp*
next
case (*5 a b t1 t2 xs ys*)
hence *eqs-old*: ∀ (*t1*,*t2*) ∈ *set* ((*t1*, *swap* [(*a*, *b*)] *t2*) # *xs*). *nabla1* ⊢ *subst* (*s1* · []) *t1* ≈ *subst* (*s1* · []) *t2*
and *fresh*: ∀ (*a*,*t*) ∈ *set* ((*a*, *t2*) # *ys*). *nabla1* ⊢ *a* ♯ *subst* (*s1* · []) *t*
using *all-solutions-def* **by** *simp-all*
hence *nabla1* ⊢ *subst* (*s1* · []) *t1* ≈ *swap* [(*a*, *b*)] (*subst* (*s1* · []) *t2*)
nabla1 ⊢ *a* ♯ *subst* (*s1* · []) *t2*
using *subst-swap-comm*[*of* ‹(*s1* · [])› ‹[(*a*, *b*)]› *t2*] **by** *simp-all*
hence *nabla1* ⊢ *subst* (*s1* · []) (*Abst a t1*) ≈ *subst* (*s1* · []) (*Abst b t2*)

using *equ-abst-ab*[*OF* 5(1) ‹*nabla1* ⊢ *a* ♯ *subst* (*s1* ∙ []) *t2*›] *subst-abst* **by** *simp*
with *eqs-old* **have** *eqs*:
∀ (*t1*,*t2*) ∈ *set* ((*Abst a t1*, *Abst b t2*) # *xs*). *nabla1* ⊢ *subst* (*s1* ∙ []) *t1* ≈ *subst* (*s1* ∙ []) *t2*
by *auto*
from *fresh eqs* **show** *?case*
using *all-solutions-def* **by** *simp*
next
case (*6 a xs ys*)
hence ∀ (*t1*,*t2*) ∈ *set xs*. *nabla1* ⊢ *subst* (*s1* ∙ []) *t1* ≈ *subst* (*s1* ∙ []) *t2*
and *fresh*: ∀ (*a*,*t*) ∈ *set ys*. *nabla1* ⊢ *a* ♯ *subst* (*s1* ∙ []) *t*
using *all-solutions-def* **by** *simp-all*
hence *eqs*:
∀ (*t1*,*t2*) ∈ *set* ((*Atom a*, *Atom a*) # *xs*). *nabla1* ⊢ *subst* (*s1* ∙ []) *t1* ≈ *subst* (*s1* ∙ []) *t2*
using *subst-unit*[*of* ‹(*s1* ∙ [])›] *equ-refl*[*of nabla1 Unit*] **by** *auto*
from *fresh eqs* **show** *?case*
using *all-solutions-def* **by** *simp*
next
case (*7 pi1 X pi2 xs ys*)
hence *eqs-old*: ∀ (*t1*,*t2*) ∈ *set xs*. *nabla1* ⊢ *subst* (*s1* ∙ []) *t1* ≈ *subst* (*s1* ∙ []) *t2*
and *fresh-old*:
∀ (*a*,*t*) ∈ *set* (*map* (λ*a*. (*a*, *Susp* [] *X*)) (*ds-list pi1 pi2*) @ *ys*). *nabla1* ⊢ *a* ♯ *subst* (*s1* ∙ []) *t*
using *all-solutions-def* **by** *simp-all*
hence ∀ *c*∈ *set* (*ds-list pi1 pi2*). *nabla1* ⊢ *c* ♯ *subst* (*s1* ∙ []) (*Susp* [] *X*)
by (*auto split*: *prod.splits*)
hence ∀ *c* ∈ *ds pi1 pi2*. *nabla1* ⊢ *c* ♯ *subst* (*s1* ∙ []) (*Susp* [] *X*)
using *ds-list-equ-ds* **by** *simp*
hence ∀ *c* ∈ *ds pi1 pi2*. *nabla1* ⊢ *c* ♯ (*look-up X* (*s1* ∙ []))
using *subst-susp*[*of* ‹(*s1* ∙ [])› ‹[]› *X*] *swap-id* **by** *simp*
hence *nabla1* ⊢ *subst* (*s1* ∙ []) (*Susp pi1 X*) ≈ *subst* (*s1* ∙ []) (*Susp pi2 X*)
using *equ-equiv-pi subst-susp*[*of* ‹(*s1* ∙ [])› - *X*] **by** *auto*
with *eqs-old fresh-old* **have**
eqs: ∀ (*t1*,*t2*) ∈ *set* ((*Susp pi1 X*, *Susp pi2 X*)#*xs*).
nabla1 ⊢ *subst* (*s1* ∙ []) *t1* ≈ *subst* (*s1* ∙ []) *t2*
and *fresh*: ∀ (*a*,*t*) ∈ *set ys*. *nabla1* ⊢ *a* ♯ *subst* (*s1* ∙ []) *t*
by *simp-all*
then show *?case*
using *all-solutions-def* **by** *simp*
next
case (*8 X t′ pi xs ys*)
hence (*nabla1*, *s1* ∙ [(*X*, *swap* (*rev pi*) *t′*)]) ∈ *U* (*xs*, *ys*)
using *problem-subst-comm* **by** *simp*
hence *eqs-old*:
∀ (*t1*,*t2*) ∈ *set xs*.
nabla1 ⊢ *subst* (*s1* ∙ [(*X*, *swap* (*rev pi*) *t′*)]) *t1* ≈ *subst* (*s1* ∙ [(*X*, *swap* (*rev pi*) *t′*)]) *t2*
and *fresh*: ∀ (*a*,*t*) ∈ *set ys*. *nabla1* ⊢ *a* ♯ *subst* (*s1* ∙ [(*X*, *swap* (*rev pi*) *t′*)]) *t*

using *all-solutions-def* **by** *simp-all*

have *subst* (*s1* · [(*X*, *swap* (*rev pi*) *t'*)]) (*Susp pi X*) = *subst s1* (*swap pi* (*swap* (*rev pi*) *t'*))
using *subst-susp subst-swap-comm subst-comp-expand eq-fst-iff look-up.simps*(*2*) *snd-eqD* **by** *metis*
also have ... = *swap pi* (*swap* (*rev pi*) (*subst s1 t'*))
using *subst-swap-comm* **by** *simp*
hence *nabla1* ⊢ *subst* (*s1* · [(*X*, *swap* (*rev pi*) *t'*)]) (*Susp pi X*) ≈ *subst s1 t'*
using *equ-refl calculation rev-pi-pi-equ*[*of nabla1 pi* ‹*subst s1 t'*›]
equ-trans[*of nabla1* ‹*subst* (*s1* · [(*X*, *swap* (*rev pi*) *t'*)]) (*Susp pi X*)›
‹*swap* (*rev pi*) (*swap pi* (*subst s1 t'*))› ‹*subst s1 t'*›]
swap-inv-side **by** *auto*

with *8 eqs-old*
have *eqs*: ∀ (*t1*,*t2*) ∈ *set* ((*Susp pi X*, *t'*)#*xs*).
nabla1 ⊢ *subst* (*s1* · [(*X*, *swap* (*rev pi*) *t'*)]) *t1* ≈ *subst* (*s1* · [(*X*, *swap* (*rev pi*) *t'*)]) *t2*
using *subst-comp-expand subst-not-occurs* **by** *simp*

from *fresh eqs* **show** *?case*
using *all-solutions-def* **by** *simp*
next
case (*9 X t' pi xs ys*)
hence (*nabla1*, *s1* · [(*X*, *swap* (*rev pi*) *t'*)]) ∈ *U* (*xs*, *ys*)
using *problem-subst-comm* **by** *simp*
hence *eqs-old*:
∀ (*t1*,*t2*) ∈ *set xs*.
nabla1 ⊢ *subst* (*s1* · [(*X*, *swap* (*rev pi*) *t'*)]) *t1* ≈ *subst* (*s1* · [(*X*, *swap* (*rev pi*) *t'*)]) *t2*
and *fresh*: ∀ (*a*,*t*) ∈ *set ys*. *nabla1* ⊢ *a* ♯ *subst* (*s1* · [(*X*, *swap* (*rev pi*) *t'*)]) *t*
using *all-solutions-def* **by** *simp-all*

have *subst* (*s1* · [(*X*, *swap* (*rev pi*) *t'*)]) (*Susp pi X*) = *subst s1* (*swap pi* (*swap* (*rev pi*) *t'*))
using *subst-susp subst-swap-comm subst-comp-expand eq-fst-iff look-up.simps*(*2*) *snd-eqD* **by** *metis*
also have ... = *swap pi* (*swap* (*rev pi*) (*subst s1 t'*))
using *subst-swap-comm* **by** *simp*
hence *nabla1* ⊢ *subst* (*s1* · [(*X*, *swap* (*rev pi*) *t'*)]) (*Susp pi X*) ≈ *subst s1 t'*
using *equ-refl calculation rev-pi-pi-equ*[*of nabla1 pi* ‹*subst s1 t'*›]
equ-trans[*of nabla1* ‹*subst* (*s1* · [(*X*, *swap* (*rev pi*) *t'*)]) (*Susp pi X*)›
‹*swap* (*rev pi*) (*swap pi* (*subst s1 t'*))› ‹*subst s1 t'*›]
swap-inv-side **by** *auto*
hence *nabla1* ⊢ *subst s1 t'* ≈ *subst* (*s1* · [(*X*, *swap* (*rev pi*) *t'*)]) (*Susp pi X*)
using *equ-symm* **by** *simp*

with *9 eqs-old*
have *eqs*: ∀ (*t1*,*t2*) ∈ *set* ((*t'*, *Susp pi X*)#*xs*).

nabla1 ⊢ *subst* (*s1* ∙ [(*X*, *swap* (*rev pi*) *t'*)]) *t1* ≈ *subst* (*s1* ∙ [(*X*, *swap* (*rev pi*) *t'*)]) *t2*
using *subst-comp-expand subst-not-occurs* **by** *simp*

from *fresh eqs* **show** *?case*
using *all-solutions-def* **by** *simp*
qed

lemma *P1-to-P2-cred*:
assumes (*nabla1*,*s1*)∈ *U P1*
and *P1* ⊢*nabla2* → *P2*
shows (*nabla1*,*s1*)∈ *U P2* ∧ (*nabla1*⊨(*subst s1*) *nabla2*)
using *assms*
proof(*induction arbitrary*: *nabla1 s1 rule*: *c-red.induct*[*OF* ‹*P1* ⊢*nabla2*→ *P2*›])
case (*2 a t1 t2 ys*)
hence *fresh-old*: ∀ (*a*,*t*) ∈ *set* ((*a*, *Paar t1 t2*) # *ys*). *nabla1* ⊢ *a* ♯ *subst s1 t*
and *eqs*: ∀ (*t1*,*t2*) ∈ *set* []. *nabla1* ⊢ *subst s1 t1* ≈ *subst s1 t2*
using *all-solutions-def* **by** *simp-all*
hence *nabla1* ⊢ *a* ♯ *subst s1* (*Paar t1 t2*)
by *simp*
hence *nabla1* ⊢ *a* ♯ *subst s1 t1 nabla1* ⊢ *a* ♯ *subst s1 t2*
using *Fresh-elims*(*5*)[*of nabla1 a* ‹*subst s1 t1*›
‹*subst s1 t2*›] **by** *auto*
with *fresh-old* **have**
fresh: ∀ (*a*,*t*) ∈ *set* ((*a*, *t1*) # (*a*, *t2*) # *ys*). *nabla1* ⊢ *a* ♯ *subst s1 t*
by *auto*
from *fresh eqs*
show *?case*
using *all-solutions-def ext-subst-def* **by** *simp*
next
case (*3 a f t ys*)
hence *fresh-old*: ∀ (*a*,*t*) ∈ *set* ((*a*, *Func f t*) # *ys*). *nabla1* ⊢ *a* ♯ *subst s1 t*
and *eqs*: ∀ (*t1*,*t2*) ∈ *set* []. *nabla1* ⊢ *subst s1 t1* ≈ *subst s1 t2*
using *all-solutions-def* **by** *simp-all*
hence *nabla1* ⊢ *a* ♯ *subst s1* (*Func f t*)
by *simp*
hence *nabla1* ⊢ *a* ♯ *subst s1 t*
using *Fresh-elims*(*6*) **by** *auto*
with *fresh-old* **have**
fresh: ∀ (*a*,*t*) ∈ *set* ((*a*, *t*) # *ys*). *nabla1* ⊢ *a* ♯ *subst s1 t*
by *auto*
from *fresh eqs*
show *?case*
using *all-solutions-def ext-subst-def* **by** *simp*
next
case (*5 a b t ys*)
hence *fresh-old*: ∀ (*a*,*t'*) ∈ *set* ((*a*, *Abst b t*) # *ys*). *nabla1* ⊢ *a* ♯ *subst s1 t'*
and *eqs*: ∀ (*t1*,*t2*) ∈ *set* []. *nabla1* ⊢ *subst s1 t1* ≈ *subst s1 t2*

using *all-solutions-def* **by** *simp-all*
hence *nabla1* ⊢ *a* ♯ *subst s1* (*Abst b t*)
by *simp*
hence *nabla1* ⊢ *a* ♯ *subst s1 t*
using *5 Fresh-elims*(*1*)[*of nabla1 a b* ‹*subst s1 t*›] **by** *auto*
with *fresh-old* **have**
fresh: ∀ (*a*,*t'*) ∈ *set* ((*a*, *t*) # *ys*). *nabla1* ⊢ *a* ♯ *subst s1 t'*
by *auto*
from *fresh eqs*
show *?case*
using *all-solutions-def ext-subst-def* **by** *simp*
next
case (*7 b pi X ys*)
hence *fresh-old*: ∀ (*a*,*t*) ∈ *set* ((*b*, *Susp pi X*) # *ys*). *nabla1* ⊢ *a* ♯ *subst s1 t*
and *eqs*: ∀ (*t1*,*t2*) ∈ *set* []. *nabla1* ⊢ *subst s1 t1* ≈ *subst s1 t2*
using *all-solutions-def* **by** *simp-all*
hence *fresh*: ∀ (*a*,*t*) ∈ *set ys*. *nabla1* ⊢ *a* ♯ *subst s1 t* **by** *simp*
from *fresh-old* **have**
nabla1 ⊢ *b* ♯ *subst s1* (*Susp pi X*) **by** *simp*
hence *nabla1* ⊢ *swapas* (*rev pi*) *b* ♯ *subst s1* (*Susp* [] *X*)
using *fresh-swap-left subst-susp* **by** *simp*
hence *ext*: *nabla1* ⊨ (*subst s1*) {(*swapas* (*rev pi*) *b*, *X*)}
using *ext-subst-def* **by** *simp*
from *fresh eqs ext*
show *?case* **using** *all-solutions-def* **by** *simp*
qed (*auto simp add*: *all-solutions-def ext-subst-def*)

lemma *P1-from-P2-cred*:
assumes (*nabla1*,*s1*)∈*U P2*
and *P1* ⊢*nabla2*→ *P2* **and** *nabla3*⊨(*subst s1*) *nabla2*
shows (*nabla1*∪*nabla3*,*s1*)∈ *U P1*
using *assms*
proof(*induction arbitrary*: *nabla1 s1 rule*: *c-red.induct*[*OF* ‹*P1* ⊢*nabla2*→ *P2*›])
case (*2 a t1 t2 xs*)
hence *fresh-old*:
∀(*b*,*t*)∈*set* ((*a*,*t1*)#(*a*,*t2*)#*xs*). *nabla1* ⊢ *b* ♯ *subst s1 t*
and ∀ (*t1*, *t2*) ∈ *set* []. *nabla1* ⊢ *subst s1 t1* ≈ *subst s1 t2*
using *all-solutions-def* **by** *auto*
hence *weak1*: ∀(*b*,*t*)∈*set xs*. (*nabla1* ∪ *nabla3*) ⊢ *b* ♯ *subst s1 t*
∀ (*t1*, *t2*) ∈ *set* []. (*nabla1* ∪ *nabla3*) ⊢ *subst s1 t1* ≈ *subst s1 t2*
using *fresh-weak* **by** *auto*
from *fresh-old* **have** *nabla1* ⊢ *a* ♯ *subst s1 t1 nabla1* ⊢ *a* ♯ *subst s1 t2*
by *simp*+
hence *nabla1* ⊢ *a* ♯ *subst s1* (*Paar t1 t2*)
using *fresh-paar subst-paar* **by** *auto*
hence (*nabla1* ∪ *nabla3*) ⊢ *a* ♯ *subst s1* (*Paar t1 t2*)
using *fresh-weak* **by** *auto*
with *weak1*

```
  have fresh: ∀(b,t)∈set ((a,Paar t1 t2)#xs). (nabla1 ∪ nabla3) ⊢ b ♯ subst s1 t
    by simp
  with weak1(2) show ?case
    using all-solutions-def ext-subst-def by simp
next
  case (3 a f t' xs)
  hence fresh-old:
    ∀(b,t)∈set ((a,t')#xs). nabla1 ⊢ b ♯ subst s1 t
    and ∀ (t1, t2) ∈ set []. nabla1 ⊢ subst s1 t1 ≈ subst s1 t2
    using all-solutions-def by auto
  hence weak1: ∀(b,t)∈set xs. (nabla1 ∪ nabla3) ⊢ b ♯ subst s1 t
        ∀ (t1, t2) ∈ set []. (nabla1 ∪ nabla3) ⊢ subst s1 t1 ≈ subst s1 t2
    using fresh-weak by auto
  from fresh-old have nabla1 ⊢ a ♯ subst s1 t'
    by simp
  hence nabla1 ⊢ a ♯ subst s1 (Func f t')
    using fresh-paar subst-paar by auto
  hence (nabla1 ∪ nabla3) ⊢ a ♯ subst s1 (Func f t')
    using fresh-weak by auto
  with weak1
  have fresh: ∀(b,t)∈set ((a, Func f t')#xs). (nabla1 ∪ nabla3) ⊢ b ♯ subst s1 t
    by simp
  with weak1(2) show ?case
    using all-solutions-def ext-subst-def by simp
next
  case (5 a b t' xs)
   hence fresh-old:
    ∀(c,t)∈set ((a,t')#xs). nabla1 ⊢ c ♯ subst s1 t
    and ∀ (t1, t2) ∈ set []. nabla1 ⊢ subst s1 t1 ≈ subst s1 t2
     using all-solutions-def by auto
   hence weak1: ∀(c,t)∈set xs. (nabla1 ∪ nabla3) ⊢ c ♯ subst s1 t
      ∀ (t1, t2) ∈ set []. (nabla1 ∪ nabla3) ⊢ subst s1 t1 ≈ subst s1 t2
     using fresh-weak by auto
   from fresh-old have nabla1 ⊢ a ♯ subst s1 t'
     by simp
   hence nabla1 ⊢ a ♯ subst s1 (Abst b t')
   using fresh-abst-ab[OF ‹nabla1 ⊢ a ♯ subst s1 t'› 5(1)] subst-abst by auto
   hence (nabla1 ∪ nabla3) ⊢ a ♯ subst s1 (Abst b t')
     using fresh-weak by auto
   with weak1
   have fresh: ∀(b,t)∈set ((a, Abst b t')#xs). (nabla1 ∪ nabla3) ⊢ b ♯ subst s1 t
     by simp
   with weak1(2) show ?case
     using all-solutions-def ext-subst-def by simp
next
  case (7 b pi X xs)
  hence fresh-old: ∀ (a,t) ∈ set xs. nabla1 ⊢ a ♯ subst s1 t
    and eqs: ∀ (t1,t2) ∈ set []. nabla1 ⊢ subst s1 t1 ≈ subst s1 t2
    using all-solutions-def by simp-all
```

hence *weak1*: $\forall (a,t) \in set\ xs.\ (nabla1 \cup nabla3) \vdash a \sharp subst\ s1\ t$
$\forall\ (t1,\ t2) \in set\ [].\ (nabla1 \cup nabla3) \vdash subst\ s1\ t1 \approx subst\ s1\ t2$
using *fresh-weak* **by** *auto*
from *7* **have** $nabla3 \vdash swapas\ (rev\ pi)\ b \sharp subst\ s1\ (Susp\ []\ X)$
using *ext-subst-def* **by** *auto*
hence $nabla3 \vdash b \sharp subst\ s1\ (Susp\ pi\ X)$
using *fresh-swap-right subst-susp* **by** *auto*
hence *weak2*: $(nabla1 \cup nabla3) \vdash b \sharp subst\ s1\ (Susp\ pi\ X)$
using *fresh-weak*[*of nabla3 b - nabla1*] *Un-commute*[*of nabla3 nabla1*] **by** *auto*
with *weak1*(*1*)
have $\forall (a,t) \in set\ ((b,Susp\ pi\ X)\#xs).\ (nabla1 \cup nabla3) \vdash a \sharp subst\ s1\ t$
by *simp*
with *weak2* **show** *?case*
using *ext-subst-def all-solutions-def* **by** *simp*
qed (*auto simp add*: *ext-subst-def all-solutions-def fresh-weak*)

lemma *P1-to-P2-red-plus1*:
assumes $P1 \models (nabla,s) \Rightarrow P2$
and $(nabla1,s1) \in U\ P1$
shows $(nabla1,s1) \in U\ P2$
using *assms*
proof(*induction P1*≡*P1 nabla*≡(*nabla*, *s*) *P2*≡*P2 arbitrary*: *s nabla1 nabla s1 P1 P2 rule*: *red-plus.induct*)
case (*sred-single P1 s P2 nabla1 s1*)
have $P1 \vdash s \rightsquigarrow P2\ (nabla1,\ s1) \in U\ P1$ **by** *fact+*
then show $(nabla1,\ s1) \in U\ P2$
using *P1-to-P2-sred* **by** *blast*
next
case (*cred-single P1 nabla1 P2 nabla2 s1*)
have $P1 \vdash nabla1 \rightarrow P2\ (nabla2,\ s1) \in U\ P1$ **by** *fact+*
then show $(nabla2,\ s1) \in U\ P2$
using *P1-to-P2-cred* **by** *blast*
next
case (*sred-step P1 s P2 nabla2 s2 P3 nabla1 s1*)
have $P1 \vdash s \rightsquigarrow P2$ **and** $(nabla1,\ s1) \in U\ P1$ **by** *fact+*
then have $(nabla1,\ s1) \in U\ P2$ **using** *P1-to-P2-sred* **by** *blast*
moreover
have *IH*: $(nabla1,\ s1) \in U\ P2 \Longrightarrow (nabla1,\ s1) \in U\ P3$ **by** *fact*
ultimately
show $(nabla1,\ s1) \in U\ P3$ **by** *simp*
next
case (*cred-step P1 nabla1 P2 nabla2 P3 nabla3 s1*)
have $P1 \vdash nabla1 \rightarrow P2$ **and** $(nabla3,\ s1) \in U\ P1$ **by** *fact+*
then have $(nabla3,\ s1) \in U\ P2$ **using** *P1-to-P2-cred* **by** *blast*
moreover
have *IH*: $(nabla3,\ s1) \in U\ P2 \Longrightarrow (nabla3,\ s1) \in U\ P3$ **by** *fact*

ultimately show (*nabla3*, *s1*) ∈ *U P3* **by** *simp*
qed

lemma *P1-to-P2-red-plus3*:
assumes *P1* ⊨ (*nabla*,*s*) ⇒ *P2*
and (*nabla1*,*s1*)∈ *U P1*
shows *nabla1*⊨(*subst s1*) *nabla*
using *assms*
proof(*induction P1*≡*P1 nabla*≡(*nabla*, *s*) *P2*≡*P2 arbitrary*: *s nabla1 nabla s1 P1 P2 rule*: *red-plus.induct*)
case (*sred-single P1 s P2 nabla1 s1*)
then show *nabla1* ⊨ (*subst s1*) {} **by** (*simp add*: *ext-subst-def*)
next
case (*cred-single P1 nabla1 P2 nabla2 s1*)
have *P1* ⊢ *nabla1* → *P2* (*nabla2*, *s1*) ∈ *U P1* **by** *fact+*
then show *nabla2* ⊨ (*subst s1*) *nabla1* **using** *P1-to-P2-cred* **by** *blast*
next
case (*sred-step P1 s P2 nabla2 s2 P3 nabla1 s1*)
have *P1* ⊢ *s* ⇝ *P2* (*nabla1*, *s1*)∈ *U P1* **by** *fact+*
then have (*nabla1*, *s1*)∈ *U P2* **using** *P1-to-P2-sred* **by** *blast*
moreover have *IH*: (*nabla1*, *s1*)∈ *U P2* ⟹ *nabla1* ⊨ (*subst s1*) *nabla2* **by** *fact*
ultimately show *nabla1* ⊨ (*subst s1*) *nabla2* **by** *simp*
next
case (*cred-step P1 nabla1 P2 nabla2 P3 nabla3 s1*)
have *IH*: (*nabla3*, *s1*) ∈ *U P2* ⟹ *nabla3* ⊨ (*subst s1*) *nabla2* **by** *fact*
have *as*: *P1* ⊢ *nabla1* → *P2* (*nabla3*, *s1*) ∈ *U P1* **by** *fact+*

from *as* **have** *nabla3* ⊨ (*subst s1*) *nabla1* **using** *P1-to-P2-cred* **by** *blast*
moreover
from *as* **have** (*nabla3*, *s1*) ∈ *U P2* **using** *P1-to-P2-cred* **by** *blast*
then have *nabla3* ⊨ (*subst s1*) *nabla2* **using** *IH* **by** *blast*
ultimately show *nabla3* ⊨ (*subst s1*) (*nabla2* ∪ *nabla1*)
by(*auto simp add*: *ext-subst-def*)
qed

lemma *P1-to-P2-red-plus2*:
assumes *P1* ⊨ (*nabla*, *s*) ⇒ *P2*
and (*nabla1*, *s1*) ∈ *U P1*
shows *nabla1* ⊨ *subst* (*s1* · *s*) ≈ *subst s1*
using *assms*
proof(*induction P1*≡*P1 nablas*≡(*nabla*, *s*) *P2*≡*P2 arbitrary*: *s nabla1 nabla s1 P1 P2 rule*: *red-plus.induct*)
case (*sred-single P1 s P2 nabla1 s1*)
have *P1* ⊢*s* ⇝*P2* (*nabla1*, *s1*) ∈ *U P1* **by** *fact+*
then show *nabla1* ⊨ *subst* (*s1* · *s*) ≈*subst s1*
using *P1-to-P2-sred* **by** *blast*
next
case (*cred-single P1 nabla1 P2 nabla2 s1*)

show *nabla2* $\models$ *subst* (*s1* $\cdot$ []) $\approx$*subst s1*
by (*simp add*: *subst-equ-refl*)
next
case (*sred-step P1 s P2 nabla2 s2 P3 nabla1 s1*)
have *IH*: (*nabla1*, *s1*) $\in$ *U P2* $\Longrightarrow$ *nabla1* $\models$ *subst* (*s1* $\cdot$ *s2*) $\approx$*subst s1* **by** *fact*
have *as*: *P1* $\vdash$*s* $\rightsquigarrow$*P2* (*nabla1*, *s1*) $\in$ *U P1* **by** *fact+*

from *as* **have** (*nabla1*, *s1*) $\in$ *U P2* **using** *P1-to-P2-sred* **by** *blast*
with *IH* **have** *nabla1* $\models$ *subst* (*s1* $\cdot$ *s2*) $\approx$ *subst s1* **by** *blast*
then have *nabla1* $\models$ *subst* ((*s1* $\cdot$ *s2*) $\cdot$ *s*) $\approx$ *subst* (*s1* $\cdot$ *s*)
by (*simp add*: *subst-cancel-right*)
moreover
from *as* **have** *nabla1* $\models$ *subst* (*s1* $\cdot$ *s*) $\approx$*subst s1*
using *P1-to-P2-sred* **by** *blast*
ultimately
show *nabla1* $\models$ *subst* (*s1* $\cdot$ (*s2* $\cdot$ *s*)) $\approx$ *subst s1*
using *subst-assoc subst-trans* **by** *metis*
next
case (*cred-step P1 nabla1 P2 nabla2 P3*)
show *nabla1* $\models$ *subst* (*s1* $\cdot$ []) $\approx$ *subst s1*
by (*simp add*: *subst-equ-refl*)
qed

lemma *P1-from-P2-red-plus*:
assumes *P1* $\models$(*nabla*,*s*)$\Rightarrow$*P2*
and (*nabla1*,*s1*)$\in$ *U P2* **and** *nabla3*$\models$(*subst s1*)(*nabla*)
shows (*nabla1*$\cup$*nabla3*,(*s1* $\cdot$ *s*))$\in$ *U P1*
using *assms*
proof(*induction P1* $\equiv$ *P1 nablas*$\equiv$(*nabla*, *s*) *P2*$\equiv$*P2 arbitrary*: *s nabla1 nabla nabla3 s1 P1 P2 rule*: *red-plus.induct*)
case (*sred-single P1 s P2 nabla1 nabla3*)
have *P1* $\vdash$ *s* $\rightsquigarrow$ *P2* (*nabla1*, *s1*) $\in$ *U P2* **by** *fact+*
then have (*nabla1*, *s1* $\cdot$ *s*) $\in$ *U P1*
using *P1-from-P2-sred* **by** *simp*
then show (*nabla1* $\cup$ *nabla3*, *s1* $\cdot$ *s*) $\in$ *U P1*
using *P1-from-P2-sred solution-weak* **by** *simp*
next
case (*cred-single P1 nabla P2 nabla1 nabla3*)
have *P1* $\vdash$ *nabla* $\rightarrow$ *P2*
(*nabla1*, *s1*) $\in$ *U P2*
nabla3 $\models$ (*subst s1*) *nabla* **by** *fact+*
hence (*nabla1* $\cup$ *nabla3*, *s1*) $\in$ *U P1*
using *P1-from-P2-cred* **by** *simp*
then show (*nabla1* $\cup$ *nabla3*, *s1* $\cdot$ []) $\in$ *U P1*
using *solution-comp-id* **by** *auto*
next
case (*sred-step P1 s P2 nabla2 s2 P3 nabla1 nabla3*)
have *IH*: (*nabla1*, *s1*) $\in$ *U P3* $\Longrightarrow$
nabla3 $\models$ (*subst s1*) *nabla2* $\Longrightarrow$ (*nabla1* $\cup$ *nabla3*, *s1* $\cdot$ *s2*) $\in$ *U P2* **by** *fact+*

```
  have as: (nabla1, s1) ∈ U P3
    nabla3 ⊨ (subst s1) nabla2
    P1 ⊢ s ⇝ P2 by fact+
  hence (nabla1 ∪ nabla3, s1 · s2) ∈ U P2
    using IH by simp
  hence (nabla1 ∪ nabla3, (s1 · s2) · s) ∈ U P1
    using as(3)
    by (simp add: P1-from-P2-sred)
  then show (nabla1 ∪ nabla3, s1 · (s2 · s)) ∈ U P1
    using solutions-subst-assoc by simp
next
  case (cred-step P1 nabla P2 nabla2 P3)
  have IH: (nabla1, s1) ∈ U P3 ⟹
     nabla3 ⊨ (subst s1) nabla2 ⟹ (nabla1 ∪ nabla3, s1 · []) ∈ U P2 by fact
  have as: P1 ⊢ nabla → P2
          (nabla1, s1) ∈ U P3
          nabla3 ⊨ (subst s1) (nabla2 ∪ nabla) by fact+
  have ext-substs: nabla3 ⊨ (subst s1) (nabla2)
    nabla3 ⊨ (subst s1) nabla
    using ext-subst-strong[OF as(3)] by simp+
  hence a: (nabla1 ∪ nabla3, s1 · []) ∈ U P2
    using IH as(2) by simp
  hence (nabla1 ∪ nabla3 ∪ nabla3, s1 · []) ∈ U P1
    using as(1) ext-substs(2) P1-from-P2-cred solution-comp-id by blast
  then show (nabla1 ∪ nabla3, s1 · []) ∈ U P1
    unfolding all-solutions-def using Un-absorb by simp
qed

lemma mgu:
  assumes P ⊨(nabla,s)⇒([],[])
  shows mgu P (nabla,s) ∧ idem (nabla,s)
proof(rule mgu-idem)
  have i: ({},[]) ∈ U ([],[])
    unfolding all-solutions-def by simp
  then show (nabla, s) ∈ U P
    using P1-from-P2-red-plus[OF assms i]
      ext-subst-id  solution-comp-id(2) by simp

  show ∀(nabla2, s2)∈ U P.  nabla2 ⊨ (subst s2) nabla ∧
                    nabla2 ⊨ subst (s2 · s) ≈ subst s2
  proof
    fix x
    assume x ∈ U P
    then show case x of (nabla2, s2) ⇒
              nabla2 ⊨ (subst s2) nabla ∧
                    nabla2 ⊨ subst (s2 · s) ≈ subst s2
    proof (cases x)
      case (Pair nabla2 s2)
      hence (nabla2,s2) ∈ U P
```

```
      using ‹x ∈ U P› by auto
    hence nabla2 ⊨ (subst s2) nabla ∧ nabla2 ⊨ subst (s2 • s) ≈ subst s2
      using assms P1-to-P2-red-plus2 P1-to-P2-red-plus3 by auto+
    with Pair show ?thesis by simp
  qed
 qed
qed


end
```

References

[1] M. Ayala-Rincón, W. de Carvalho Segundo, M. Fernández, D. Nantes-Sobrinho, and A. C. R. Oliveira. A formalisation of nominal α-equivalence with a, c, and AC function symbols. *Theor. Comput. Sci.*, 781:3–23, 2019.

[2] M. Ayala-Rincón, M. Fernández, and A. C. R. Oliveira. Completeness in PVS of a nominal unification algorithm. In M. R. F. Benevides and R. Thiemann, editors, *Proceedings of the Tenth Workshop on Logical and Semantic Frameworks, with Applications, LSFA 2015, Natal, Brazil, August 31 - September 1, 2015*, volume 323 of *Electronic Notes in Theoretical Computer Science*, pages 57–74. Elsevier, 2015.

[3] W. L. R. de Carvalho. *Nominal Equational Problems Modulo Associativity, Commutativity and Associativity-Commutativity.* PhD thesis, Graduate Program in Informatics, University of Brasília, 2019. in English.

[4] A. C. R. Oliveira. *Unification, Confluence, and Intersection Types for Nominal Rewriting Systems.* PhD thesis, Graduate Program in Informatics, University of Brasília, 2016. in English.

[5] C. Urban. Nominal unification revisited. In M. Fernández, editor, *Proceedings 24th International Workshop on Unification, UNIF 2010, Edinburgh, United Kingdom, 14th July 2010*, volume 42 of *EPTCS*, pages 1–11, 2010.

[6] C. Urban, A. M. Pitts, and M. Gabbay. Nominal unification. *Theor. Comput. Sci.*, 323(1-3):473–497, 2004.